ScintillaOS

操作系统内核设计文档



学校：武汉大学

队员：雷雨泽、杜宇恒、宋天琦

二〇二六年八月

摘　　要

ScintillaOS 是一个基于 rCore、使用 Rust 实现、面向全国大学生操作系统设计大赛 QEMU 环境的操作系统内核。系统面向 Linux ABI 用户程序，支持 RISC-V 与 LoongArch 平台；在当前配置下，RISC-V 最多支持 8 个应用 CPU，LoongArch 最多支持 12 个应用 CPU。内核提供用户态 ELF 装载、按需虚拟内存与写时复制、进程、线程、信号和 futex、VFS 与 Ext4 双盘、VirtIO 块设备与网卡、TCP/UDP 套接字，以及 eventfd、epoll、管道和 Unix 域 socketpair 等事件 I/O 能力，目标是在 glibc 用户态和 Rust/Cargo 等复杂负载下保持可运行性、并发正确性和可验证性。

ScintillaOS 这一名称源于 Scintilla。Scintilla 有“火花”之意，寓意 **ScintillaOS 并没有选择现成的完整操作系统内核优化**，而是在 rCore 底层内核提供的基础火花之上，沿着启动、内存、进程、文件系统、设备驱动与网络支持等方向逐步延展、蔓延并搭建起来。这个名字既概括了 ScintillaOS 从核心出发不断扩展的开发过程，也表达了**团队希望能够从教学级基础内核开始，真正进行一次类 linux 操作系统内核的完整开发**，打磨出可运行、可扩展、可评测的竞赛内核的目标，从而深度锻炼团队的整体计算机系统工程能力。

本报告在 1-10 章具体说明了系统的总体架构和关键模块设计，并在第 11-12 章详细记录了本操作系统的开发流程，开发时所遇到的典型问题与团队所采用的具体解决方案。此外，本报告在附录中依照大赛要求提供了 AI 使用说明。截至 2026 年 6 月 29 日 12:00，ScintillaOS 在全国大学生操作系统设计大赛平台初赛上取得了 1906 分，在排行榜上位列 47 名。截至 2026 年 8 月 12 日 12:00，ScintillaOS 在全国大学生操作系统设计大赛平台决赛上取得了 709 分，在排行榜上位列 10 名。

关键词: ScintillaOS；操作系统内核；Rust；RISC-V；Loongarch；rCore

评测完成

开始时间：2026-06-29 09:48:47.034337+08:00

结束时间：2026-06-29 11:56:02.227621+08:00

总分：1906.5102347009993

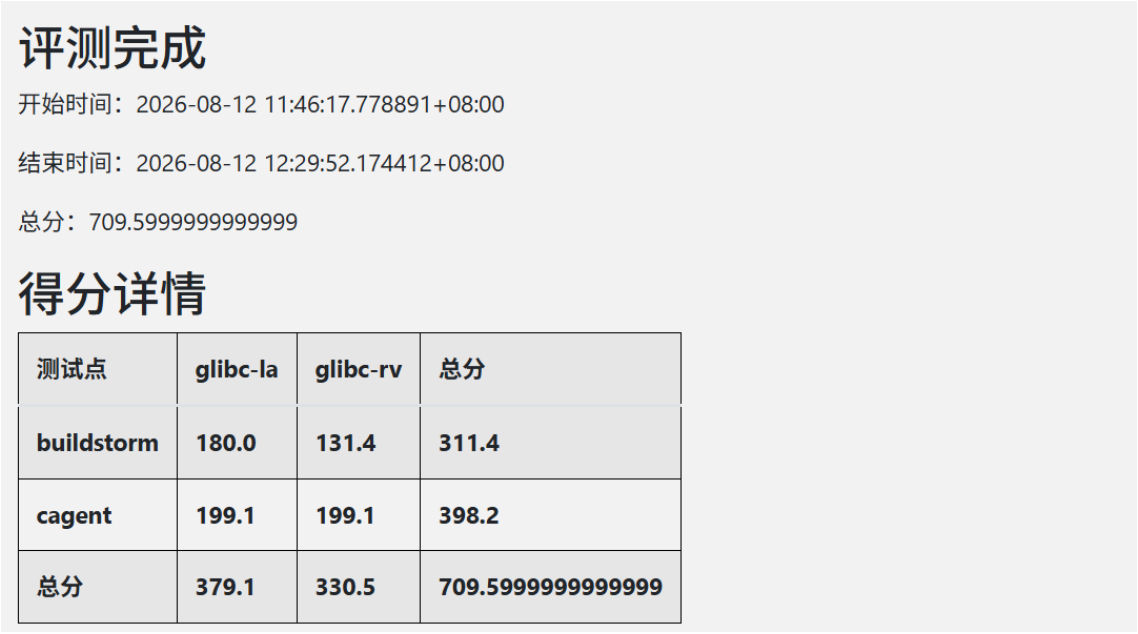
得分详情

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	54	54	54	54	216
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	20.0	20.0	20.0	20.0	80.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	30.98880191481142	0.0	29.570856568964814	27.136759589774613	87.69641807355084
libctest	-	-	215	215	430
lmbench	45.41394127071955	45.06188963535785	46.335299714047146	46.53068761507888	183.34181823520342
ltp	1068	1068	1066	1066	4268
lua	9	9	9	9	36
netperf	0.0	0.0	0.0	0.0	0.0
总分	1329.402743185531	1298.0618896353578	1541.906156283012	1539.6674472048535	1906.5102347009993

图 1　ScintillaOS 的初赛得分情况。ScintiallaOS 通过了 basic，busybox，iozone，libcbench，libctest，lmbench，ltp，lua 等多个官方测试用例。



图 2　ScintillaOS 的初赛排名。截至 2026 年 6 月 29 日，ScintillaOS 在排行榜上位列 47 名。



评测完成

开始时间：2026-08-12 11:46:17.778891+08:00

结束时间：2026-08-12 12:29:52.174412+08:00

总分：709.5999999999999

得分详情

测试点	glibc-la	glibc-rv	总分
buildstorm	180.0	131.4	311.4
cagent	199.1	199.1	398.2
总分	379.1	330.5	709.5999999999999

图 3　ScintillaOS 的决赛得分情况。

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T2026104879910415	维塔立体	华中科技大学	46	180.0	180.0	199.1	199.1	758.2000
2	T202610617999569	每天睡满八小时	重庆邮电大学	60	180.0	180.0	199.1	199.1	758.2000
3	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
4	T2026104869910383	洗剪吹	武汉大学	77	180.0	169.3	199.1	199.1	747.5000
5	T202610459999616	PulseOS	郑州大学	19	180.0	151.3	199.1	199.1	729.5000
6	T2026181239910993	Unicus	哈尔滨工业大学（深圳）	36	168.5	157.9	199.1	199.1	724.6000
7	T2026104869911460	最小可运行	武汉大学	28	180.0	140.2	199.1	199.1	718.4000
8	T2026104869910054	猛火烧烤队	武汉大学	51	180.0	138.9	199.1	199.1	717.1000
9	T202610486999578	NexusCore	武汉大学	52	180.0	132.0	199.1	199.1	710.2000
10	T202610486999873	我依旧是世一队	武汉大学	19	180.0	131.4	199.1	199.1	709.6000
11	T2026104869910397	CoreCraft	武汉大学	22	161.8	149.2	199.1	199.1	709.2000
12	T2026103369910030	拼好核	杭州电子科技大学	61	156.3	130.1	199.1	199.1	684.6000
13	T2026103369910596	CosmOS	杭州电子科技大学	48	152.6	112.0	199.1	199.1	662.8000
14	T202610336999669	StellarOS	杭州电子科技大学	12	20.0	180.0	199.1	199.1	598.2000
15	T2026181239910972	卄世界电子开发部	哈尔滨工业大学（深圳）	44	134.4	60.0	199.1	199.1	592.6000

图 4　ScintillaOS 的决赛排名。截至 2026 年 8 月 12 日，ScintillaOS 在排行榜上位列 10 名。

目　录

1 系统概述 **1**

1.1 名称含义 1
1.2 设计目标与运行环境 1
1.3 总体架构 2
1.4 组件边界与设计取舍 3
1.5 模块协作机制与 Rust 语言特性 3
1.6 系统调用接口设计 5
1.7 代码组织 5
1.8 启动与执行流程 6
1.9 参考与改进 6

2 内核关键设计与实现 **7**

2.1 rCore 基础 7
2.2 ScintillaOS: 从裸机启动到用户态执行 8
2.3 进程模型与调度：以内核对象组织用户程序生命周期 11
2.4 虚拟内存：同时支撑隔离、装载与按需分配 13
2.5 VFS 与文件描述符：用统一抽象连接磁盘、管道、设备和 proc . . 15
2.6 Linux 系统调用兼容：在 ABI 边界上保持语义一致 17
2.7 并发同步与资源管理 18
2.8 平台与设备适配：在 QEMU virt 上验证完整内核路径 19
2.9 小结 20

3 SMP 调度与并发同步设计 **21**

3.1 设计目标与核心不变量 21
3.2 CPU 本地状态与多核启动协议 22
3.3 每 CPU 就绪队列、亲和性与任务选择 23
3.4 任务状态机与上下文交接 24
3.5 统一阻塞、二次检查与并发唤醒 25
3.6 IPI、远端 TLB 刷新与跨核可见性 26
3.7 领域锁、锁顺序与诊断机制 26

3.8 设计边界与小结 28

4 内存管理模块 29

4.1 虚拟内存机制 29
4.2 物理内存分配 30
4.3 ELF 装载与用户栈 32
4.4 缺页、按需分页与写时复制 33
4.5 共享地址空间与堆管理 34

5 进程管理模块 36

5.1 核心数据结构 36
5.2 PID 管理与调度器 38
5.3 创建、切换与执行 38
5.4 进程层级、组与资源边界 39
5.5 进程生命周期 40
5.6 信号机制 40
5.7 退出、回收与等待 41

6 文件系统模块 44

6.1 虚拟文件系统架构 44
6.2 核心对象与对象关系 44
6.3 路径解析与文件打开 46
6.4 Ext4 与双盘设计 48
6.5 挂载、卸载与一致性 49
6.6 特殊文件和 IPC 50

7 信号处理模块 51

7.1 信号发送与待处理状态 51
7.2 信号递送时机 52
7.3 信号处理的限制 53

8 系统调用与用户态支持 54

8.1 系统调用接口 54
8.2 系统调用支持 55
8.3 用户态运行时 56

9 网络模块 57

9.1 设计目标与分层架构 57
9.2 设备、协议栈与轮询模型 58

9.3 Socket fd 与端点状态 . . . 58
9.4 TCP 与 UDP 传输路径 . . . 58
9.5 阻塞 I/O、轮询与跨核唤醒 . . . 59
9.6 本地 Unix 域通信与资源回收 . . . 60
9.7 支持范围与工程边界 . . . 60

10 决赛负载适配 62
10.1 负载特征与适配目标 . . . 62
10.2 双盘评测执行链 . . . 62
10.3 多进程、多线程与 SMP 资源调度 . . . 62
10.4 按需装载、内存容量与页缓存 . . . 63
10.5 高频文件系统操作与写回控制 . . . 63
10.6 事件接口、时间与运行时兼容 . . . 64
10.7 验证策略与当前边界 . . . 64

11 开发目标、计划与过程记录 66
11.1 开发目标与范围 . . . 66
11.2 阶段计划 . . . 67
11.3 完整开发过程描述 . . . 67
11.4 团队分工协作与证据 . . . 68

12 Scintilla 技术问题及解决方案示例 70
12.1 问题记录 . . . 70
12.2 案例一：VFS、双盘路径解析与并发目录项缓存 . . . 71
12.3 案例二：多核 COW、用户态写回与远端 TLB 刷新 . . . 73
12.4 案例三：`mmap` 的边界条件与共享地址空间并发 . . . 74
12.5 案例四：SMP 阻塞唤醒与锁顺序 . . . 76

13 构建、评测与结果 79
13.1 构建与运行方法 . . . 79
13.2 测试框架 . . . 79

14 总结与展望 80
14.1 当前工作情况 . . . 80
14.2 经验总结 . . . 80
14.3 未来展望 . . . 80

附录 A AI 辅助使用说明 82

附录 B　参考资料　84

1 系统概述

1.1 名称含义

ScintillaOS 这一名称源于 Scintilla。Scintilla 有“火花”之意，寓意 ScintillaOS 并不是凭空生成的庞然整体，而是在 rCore 教学内核提供的基础火花之上，沿着启动、内存、进程、文件系统、设备驱动与网络支持等方向逐步延展、蔓延并搭建起来。这个名字既概括了 ScintillaOS 从核心执行链路出发不断扩展的开发过程，也表达了团队希望把一个教学内核的起点打磨成可运行、可测试、可继续演进的竞赛内核的目标。

1.2 设计目标与运行环境

ScintillaOS 的目标是在 QEMU 虚拟平台上提供兼容 Linux ABI 的用户程序运行环境，并接入大赛测试盘完成基础功能、文件系统、网络、性能与复杂构建测试。内核主要使用 Rust 实现，支持静态链接 ELF 的启动与运行，并验证了 glibc 用户态、动态运行时相关路径以及 Rust/Cargo 工具链负载。

ScintillaOS 采用第一代 SMP 内核设计：每个 hart 维护独立的 CPU 本地状态和就绪队列，任务通过在线 CPU 掩码、允许 CPU 集合和唯一运行所有权参与调度；空闲 CPU 可由 IPI 唤醒。当前配置中，RISC-V 平台最多启用 8 个应用 CPU，LoongArch 平台最多启用 12 个应用 CPU，较小的 `-smp` 配置仍可动态上线。RISC-V 使用 Sv39 三级页表和 4 KiB 页面；文件系统采用 Ext4，并通过 `lwext4_rust` 调用嵌入式 lwext4。设备层支持 VirtIO 块设备与 VirtIO 网卡：RISC-V 使用 MMIO 传输，LoongArch 使用 PCI 传输。系统调用层沿用 Linux ABI 编号和统一陷阱分发机制。

表 1.1　系统主要特征

特征	说明
处理器模型	每 CPU FIFO 就绪队列、CPU 本地状态、在线 CPU 掩码和 IPI 唤醒。当前配置支持 RISC-V 最多 8 个应用 CPU、LoongArch 最多 12 个应用 CPU。
体系结构	RISC-V 与 LoongArch 均为评测目标；两者均支持多核启动、调度和双盘运行，设备传输方式按平台分别适配。
地址空间	RISC-V 使用 Sv39 三级页表，页面大小为 4 KiB。地址空间支持跨核活动掩码、远端 TLB 刷新和文件页缓存等机制
文件系统	Ext4；通过 `lwext4_rust` 调用嵌入式 lwext4 库。
设备与虚拟化	QEMU Virt 平台、OpenSBI、VirtIO 块设备和 VirtIO 网卡。
接口	系统调用编号遵循 RISC-V Linux ABI，内核在陷阱处理处完成分发。普通文件、管道、socket、eventfd、epoll 与 Unix 域 socket 进入统一文件描述符命名空间。

1.3　总体架构

系统可分为用户态、系统调用与陷阱层、内核核心子系统以及设备层四个层次。图1.1概括了这些层次之间的调用关系。内核核心子系统包含 SMP 调度与同步层：任务管理器维护每 CPU 就绪队列和任务状态，领域锁保护共享资源，IPI 和远端 TLB 刷新负责跨核协作。用户程序通过系统调用进入内核，内核将请求分发给进程、内存、文件系统、信号和网络模块；时钟中断在保持调度精度的同时驱动定时器与网络轮询。

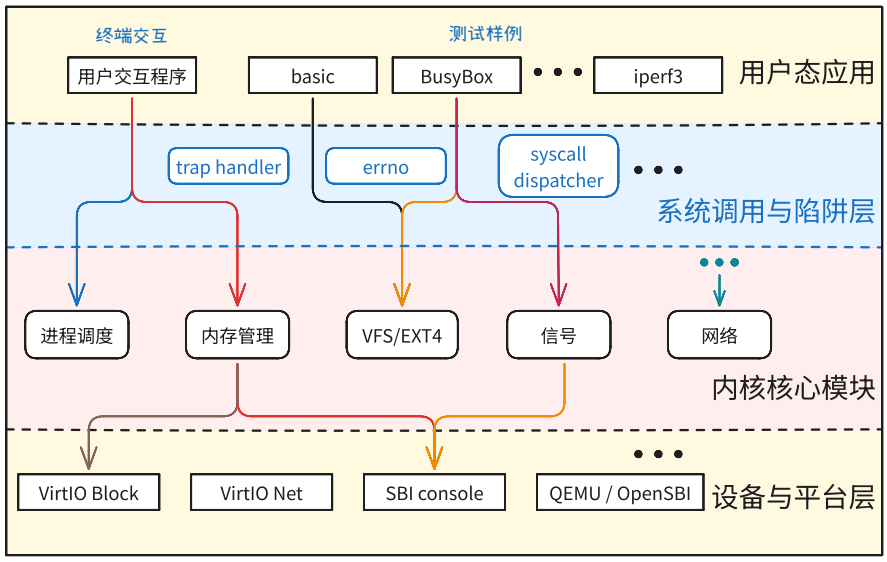


图 1.1　ScintillaOS 总体架构

1.4　组件边界与设计取舍

内核按功能划分模块，而非将全部逻辑集中在系统调用实现中。`task` 模块拥有进程状态和调度队列；`mm` 模块负责页表与地址空间；`fs` 模块将 VFS 接口下发给 Ext4；`net` 模块维护套接字及网络轮询；`drivers` 模块屏蔽 VirtIO 设备细节。系统调用层只承担参数转换、用户地址校验、权限检查和模块调用，从而使同一底层能力可以被多个系统调用复用。

ScintillaOS 已经在决赛阶段扩展为多核处理器。我们仍然将 `UPSafeCell` 保留在没有跨核共享的兼容路径中；任务表、调度器、futex、文件描述符组、地址空间、页帧、VFS、Ext4、Dentry 和设备队列等并发共享对象使用领域锁保护。锁按 CPU 本地状态、PID 注册表、futex、调度器、任务、虚拟内存、文件系统、设备等顺序获取，并要求不得持有领域锁跨越调度、阻塞 I/O、用户地址转换、页表修改或外部回调。调试与 `smp-diagnostics` 构建启用 lockdep，用于识别递归加锁、锁顺序反转和切换前遗留锁。

1.5　模块协作机制与 Rust 语言特性

ScintillaOS 围绕“任务运行一个用户程序”这一主线协作：用户态通过 `ecall` 进入内核；系统调用层校验用户参数后调用进程、内存、文件系统或网络模块；模块返回结果后由陷阱层写回寄存器；时钟中断驱动调度、定时器和网络轮询。任

务可能在不同 CPU 上执行，因此调度器必须先发布任务所有权，再完成上下文切换；页表更新必须在其他活跃 CPU 刷新 TLB 后才能回收旧页帧。

表 1.2　关键机制、引入原因与 Rust 支撑

机制	要解决的问题	设计选择	Rust 语言特性
COW、按需分页与页缓存	fork 立即复制大量页、文件映射提前读入会浪费内存	共享只读页；写缺页时复制；跨核修改页表后刷新 TLB；只读文件页进入可失效的页缓存	`FrameTracker` 的 `Drop` 回收、引用计数共享帧和受控锁域
TCB、线程组与 SMP 调度	一个任务不能被两个 CPU 同时执行，线程还可能共享 VM、fd 和信号动作	每 CPU 队列；`running_cpu` 与 `enqueued` 原子不变量；`clone` 按标志共享资源	`Arc` 共享任务、`Weak` 表示亲缘关系、原子状态配合领域锁
VFS 与 Dentry	多文件系统、挂载点和复杂构建不能依赖字符串拼接	超级块、Dentry、挂载表和 `File` trait 分层；打开后删除延迟到最后 fd 关闭	trait 统一 I/O 对象；`Arc/Weak` 管理目录树和文件对象生命周期
统一阻塞唤醒	多核下检查条件与真正睡眠之间可能丢失唤醒	`Running -> Blocking -> Blocked -> Ready`；竞争唤醒使用 `WakePending`	状态转换集中在调度器锁内，避免可变借用跨上下文切换
信号帧与 trampoline	用户处理函数返回后必须恢复完整现场，线程间还可能共享处理动作	按 RISC-V Linux ABI 构造 `siginfo/ucontext`；映射 `rt_sigreturn` trampoline	独立用户 ABI 结构与边界检查限制不安全用户地址访问
事件与 socket fd	复杂用户程序需要事件通知、Unix IPC 和网络对象统一纳管	`eventfd`、`epoll`、Unix `socketpair`、TCP/UDP 均进入 fd 命名空间	trait 对象统一关闭与 poll 生命周期；`Arc` 管理端点共享状态

1.6 系统调用接口设计

用户程序将系统调用号放入 RISC-V 的 a7 寄存器，将最多六个参数放入参数寄存器后执行 ecall。我们通过陷阱处理程序识别用户态系统调用，调用 syscall(syscall_id, [args; 6])，并将返回值写入 TrapFrame 的返回寄存器。而从代码层面来看，文件、进程和网络调用则分别分派到 syscall/fs.rs、syscall/process.rs 与 net/syscall.rs 三个文件中。

ScintillaOS 采用“ABI 解码 + 子模块实现”的分层，并扩展了支撑 glibc、pthread 和复杂构建的调用路径。除普通文件、管道、stdin/stdout、TCP/UDP socket 外，eventfd2、epoll_create1、epoll_ctl、epoll_pwait、socketpair、sendmsg 与 recvfrom 等接口也被接入分发器。

文件、管道、eventfd、epoll、Unix 域 socket 与网络 socket 均通过文件描述符表统一管理。CLONE_FILES 线程共享 fd 表时，描述符表修改由独立锁保护；exec 按 FD_CLOEXEC 关闭对应描述符，以保持多线程程序的描述符生命周期语义。

1.7 代码组织

ScintillaOS 的操作系统设计在 main 、MachineTest 和 finalsubmission 三个分支中。其中 main 分支用来开发，而 MachineTest 用来官网评测，finalsubmission 分支则用于决赛阶段的评测。此外，我们在 documents 目录下详细记录了 ScintillaOS 内核的开发流程。ScintillaOS 的具体结构则参考下表：

目录	职责
os/src	内核主体；包括启动、陷阱、进程、内存、文件系统、网络、驱动和系统调用。
user/src	用户态运行时、系统调用封装、Shell、自动评测启动程序及 SMP 定向测试程序。
lwext4_rust	lwext4 的 Rust 绑定及块设备适配层。
basic	基础系统调用测试程序与构建脚本。
documents	架构、VFS、Ext4 迁移、SMP、BuildStorm、CAgent 和测试接入过程的内部开发记录、可行性分析和技术讨论。

1.8 启动与执行流程

主 CPU 的启动路径从 `os/src/main.rs` 开始：初始化内核堆、物理帧分配器、页表、陷阱向量、根 Ext4 文件系统与设备，再装载 `/bin/initproc`。次级 hart 先初始化自己的 per-CPU 存储和 idle 上下文，随后等待主 CPU 完成全局资源初始化与首个任务进入用户态；只有在此之后才登记 online bit 并加入调度。该顺序避免次级 CPU 过早观察到未完成的调度器、页表或设备状态。

调度器从当前 CPU 的就绪队列取得任务，发布其唯一 `running_cpu` 所有权，切换页表和上下文后返回用户态。时钟中断负责调度节拍、睡眠检查和到期定时器；网络轮询使用单调硬件时钟，避免 LoongArch 的高频硬件中断将网络时间按调用次数错误推进。

1.9 参考与改进

ScintillaOS 沿用教学操作系统中“陷阱—系统调用—进程—地址空间”的基本分层，并针对大赛测试环境实现了 Linux ABI、VFS/Dentry/Mount、Ext4 双盘、COW、TCP/UDP socket、SMP 调度与同步、跨核页表一致性、事件 I/O、Unix IPC、双架构 VirtIO-Net 以及面向 glibc/Rust-Cargo 的兼容路径。

与完整通用宏内核相比，ScintillaOS 当前仍以 QEMU 上的竞赛工作负载为边界。系统已经具备 per-CPU 队列、IPI、CPU affinity 和远端 TLB 刷新，但不宣称实现成熟的负载均衡、NUMA、完整作业控制、IPv6、原始 socket 或全部 POSIX 语义。BuildStorm Q2.2 的完整复杂工程构建也仍需在目标架构、目标内存配置和干净测试盘上完成最终验证。

2 内核关键设计与实现

本章围绕内核关键模块的设计与实现展开说明。需要首先明确的是，ScintillaOS 并不是从零开始实现一个操作系统内核，而是在 rCore 教学内核的基础上继续开发。rCore 已经提供了 RISC-V 平台上的基本裸机启动流程、特权级切换、异常与中断框架、任务抽象、地址空间管理以及 Rust 编写内核的基本工程结构。我们选择基于 rCore 开发，是因为它提供了一条清晰的最小内核主线：从机器启动、进入内核、建立页表，到创建用户任务并通过系统调用返回内核。这使 ScintillaOS 能够将主要精力放在比赛要求的 Linux 兼容能力、文件系统能力和复杂资源管理能力上，而不是重复搭建最基础的内核运行框架。

在 rCore 提供的启动、陷阱、任务切换和虚拟内存框架上，ScintillaOS 围绕 Linux 应用兼容与多核执行需求进行扩展。内核以进程、线程组、地址空间和文件描述符表组织资源，支持 `fork`、`clone`、`exec`、`wait`、`mmap`、Ext4、pipe、proc、VirtIO 块设备和网络设备；同时以每 CPU 调度、领域锁、IPI、远端 TLB 刷新和统一阻塞唤醒保证这些模块在多个 CPU 上协作。Rust 的所有权、引用计数和受控 `unsafe` 边界用于降低页帧、文件对象、任务对象和设备缓冲区的生命周期风险。

比赛内核的难点不只是补充若干系统调用，还在于保持跨模块语义的一致性。例如，`fork` 同时依赖地址空间 COW、文件描述符继承和任务调度；`exec` 依赖 ELF 装载、共享线程资源收敛和 close-on-exec；`mmap` 同时涉及 VFS、文件页缓存和缺页处理；`munmap`、COW 权限变化和共享地址空间修改还需要对其他活跃 CPU 刷新 TLB。本章按照这些依赖关系说明系统的关键设计。

2.1 rCore 基础

rCore 为 ScintillaOS 提供的是一条已经打通的最小内核执行链路。它基于 RISC-V 架构和 Rust 语言实现，从固件交接、进入 S 态内核、建立栈和页表，到设置陷阱入口、创建用户任务、通过 `ecall` 返回内核，形成了操作系统从裸机启动到用户态执行的基本闭环。ScintillaOS 选择继承这条主线，是因为比赛内核的

重点并不是再次证明“内核能够启动”，而是在已有执行链路上扩展 Linux 兼容语义、文件系统、设备和资源管理能力。

rCore 的价值首先体现在边界清晰。RISC-V 的特权级、异常中断和页表机制，使用户态和内核态之间存在明确的硬件隔离；rCore 在此基础上把系统调用、缺页异常、非法指令和时钟中断统一纳入陷阱处理路径。这样，ScintillaOS 后续实现 `fork`、`exec`、`mmap`、文件访问和设备中断时，不需要重新设计用户态进入内核的方式，而是可以围绕同一个可信入口扩展不同模块的语义。

rCore 的第二个价值在于资源抽象已经具备雏形。任务上下文、地址空间、页表映射、物理页帧分配和基础文件访问共同构成了进程运行的必要条件。ScintillaOS 在这些抽象上继续向 Linux 语义靠拢：任务需要扩展为包含父子关系、退出状态和文件描述符表的进程；地址空间需要支持 COW、文件映射和按需装页；文件访问需要从简单文件系统扩展为 VFS、Ext4、pipe、proc 和设备文件。也就是说，rCore 提供的是骨架，ScintillaOS 的工作是在骨架上补齐比赛所需的兼容能力。

采用 Rust 也是继承 rCore 的重要原因。页帧、进程对象、文件对象和目录项都具有复杂生命周期，Rust 的所有权、引用计数和 RAII 能够帮助内核把大部分资源释放逻辑收束到对象边界上。不过，陷阱入口、页表切换、用户指针访问和设备 MMIO 仍然必须使用 `unsafe`。因此，ScintillaOS 没有把 Rust 理解为自动安全保证，而是沿用 rCore 的思路：让普通内核逻辑尽量保持在安全抽象中，把必须越过类型系统的部分集中在硬件交互和地址转换边界。

rCore 提供的执行链路是 RISC-V 实现的基础；ScintillaOS 通过 PolyHAL、架构相关启动代码、陷阱实现和设备传输层将相同的内核抽象扩展到 LoongArch。两种架构共享任务、内存、VFS、系统调用和网络的上层语义，而在固件交接、IPI、定时器、页表和 VirtIO 传输等硬件边界分别适配。

2.2 ScintillaOS: 从裸机启动到用户态执行

对于 ScintillaOS 而言，内核启动阶段的核心任务，是把机器从固件交接后的运行状态，逐步转换为能够运行用户程序的环境。之所以首先实现启动流程、页表、陷阱入口和第一个用户进程，是因为后续所有系统调用、进程调度、文件访问和内存管理，都依赖这些基础设施。没有物理页帧管理，就无法为用户进程构造独立地址空间；没有根文件系统，就无法装载 `/bin/initproc`；没有陷阱入口，用

户程序发生系统调用、缺页或非法访问时就无法安全回到内核。

启动路径由主 CPU 完成全局初始化：内核堆与页帧分配器建立动态分配能力，页表和陷阱向量建立地址转换与受控入口，根 Ext4 文件系统及 VirtIO 设备使内核能够读取用户程序，随后 `/bin/initproc` 被装载为首个任务。次级 hart 则先完成自己的 per-CPU 存储和 idle 上下文初始化，在主 CPU 发布全局就绪状态及首个用户任务后登记 online bit 并进入本地调度循环。这样，次级 CPU 不会在调度器、页表或设备尚未准备完成时访问共享状态。

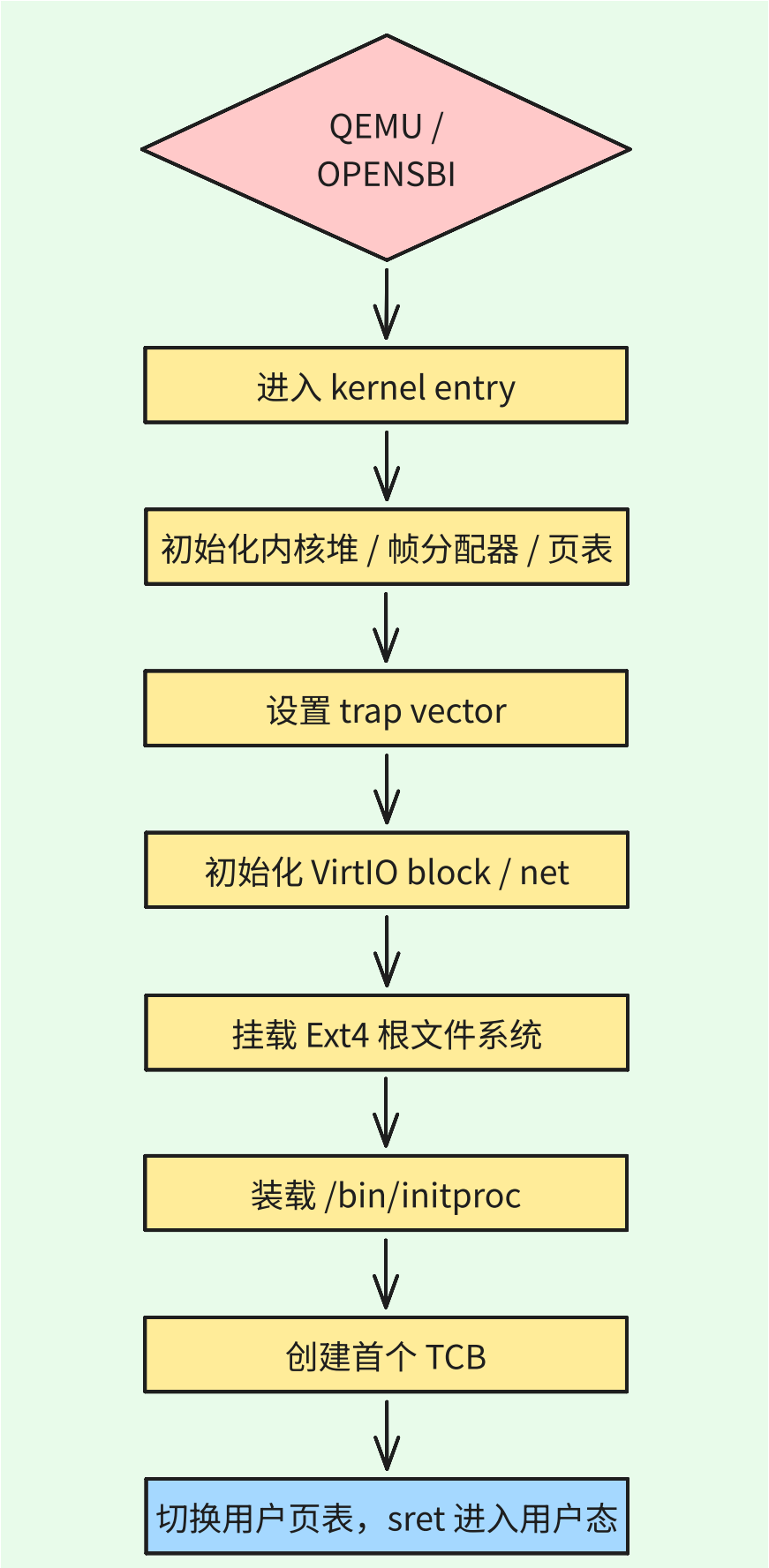


图 2.1　从内核启动到首个用户进程执行的主路径

统一陷阱入口是 ScintillaOS 后续设计的关键。用户程序不能直接调用内核函数，也不能直接访问内核资源，它只能通过硬件定义的异常入口进入内核。系统调用属于同步异常，用户程序按照 RISC-V Linux ABI 将系统调用号放入 `a7`，参数放入 `a0--a5`，执行 `ecall` 后进入内核。时钟中断和设备中断则是异步事件，可能在用户程序任意位置发生，因此内核必须保存被打断的寄存器现场，处理定时器、设备轮询或调度事件后，再恢复原任务或切换到新任务。

多核陷阱路径还必须保证每个 CPU 的上下文和用户 TrapFrame 不被其他 CPU 混用。RISC-V 在从内核恢复用户态的临界窗口内关闭本地中断，避免 timer interrupt 覆盖已发布的用户 TrapFrame；用户态中断状态仍由 TrapFrame 在最终返回指令处恢复。LoongArch 使用对应的架构陷阱、定时器和 IPI 路径，但上层调度与信号递送语义保持一致。

这里选择“统一陷阱入口 + 分发处理”的技术路线，是为了把用户态越权访问、缺页、系统调用和时钟抢占全部纳入同一个可信边界。这样做可以避免每个模块自行处理特权级切换，从而降低安全边界分散带来的错误风险。需要强调的是，Rust 能够减少普通控制流中的生命周期和资源管理错误，但陷阱入口、汇编保存现场、页表切换和用户指针访问仍然依赖架构约定与 `unsafe` 代码。因此，ScintillaOS 将 unsafe 尽量限制在硬件交互和边界转换位置，而不是让其扩散到普通业务逻辑中。

2.3 进程模型与调度：以内核对象组织用户程序生命周期

在完成从用户态进入内核的基本通路后，内核必须回答第二个问题：用户程序在内核中应当以什么形式存在。我们选择以“任务控制块 + 地址空间 + 文件描述符表 + 父子关系”作为进程抽象的核心，是因为 Linux 兼容语义中的 fork、exec、wait、exit、文件继承和资源回收，都需要围绕同一个进程对象展开。

`TaskControlBlock` 是进程资源的聚合点，并且它同时承载线程组、CPU 亲和性、唯一运行 CPU、排队状态、共享地址空间、共享文件描述符组和信号动作等状态。调度器通过 TCB 找到可运行任务；系统调用通过 TCB 访问当前地址空间、文件描述符表和工作目录；退出路径保存退出码并通知父进程或同线程组等待者；wait 路径据此回收僵尸进程。

调度器使用每 CPU FIFO 就绪队列。新任务根据允许 CPU 集合和在线 CPU

掩码选择合适队列，空闲目标 CPU 由 IPI 提示重新检查队列。每个 TCB 的 `enqueued` 与 `running_cpu` 原子状态分别约束“最多存在一个就绪队列条目”和“最多被一个 CPU 执行”；任务仅在切换回本 CPU 的 idle 上下文、寄存器现场已完成保存后，才能再次发布为可运行状态。该设计优先保证任务所有权和阻塞唤醒的正确性，不以 Linux CFS 或强制迁移式负载均衡为目标。

信号递送和优先唤醒不能仅依据一次任务状态读取决定是否入队。内核按 `SchedulerLock -> TaskLock` 的顺序重新核对目标：仍为 `Ready` 时只提升既有队列项；已变为 `Running` 时不创建第二个队列引用，而是向实际运行 CPU 请求重新调度，使其尽快观察 pending signal。该规则消除了“任务从 Ready 被另一 CPU 取走后又被重复入队”的所有权竞态。

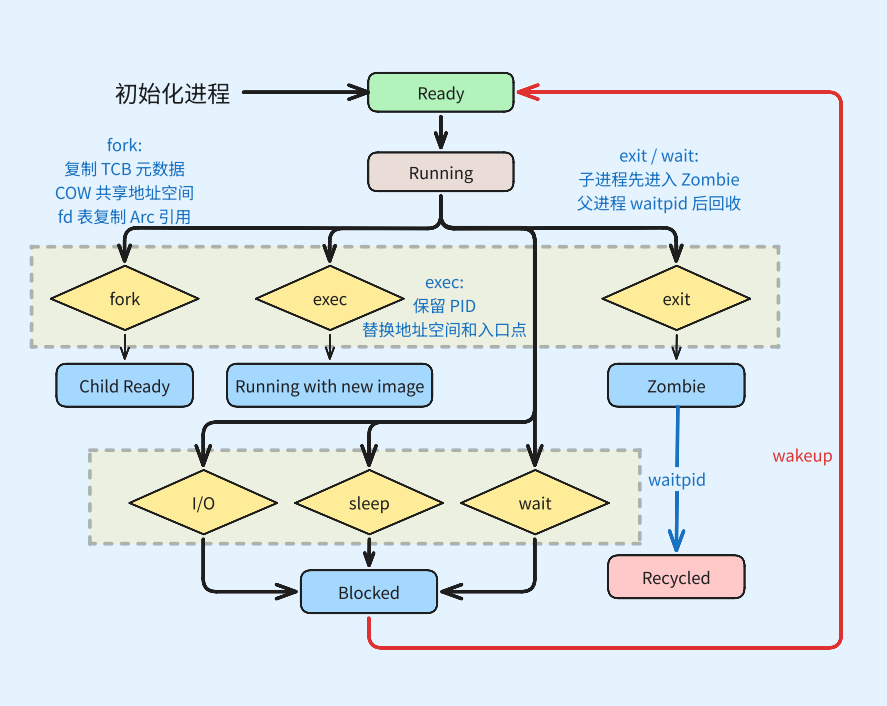


图 2.2　进程生命周期与资源语义

`fork`、`exec` 和 `wait` 体现了进程模型与其他模块之间的协作关系。`fork` 要求父子进程从同一个用户现场继续执行，但二者拥有不同返回值和独立进程身份。为了避免立即复制整个地址空间，内存模块为 fork 提供 COW 映射，使父子进程先共享只读物理页，只有发生写入时才复制页面。为了保持文件继承语义，文件描述符表复制的是 `Arc<dyn File>` 引用，而不是复制文件对象本身。这样父子进程可以共享打开文件状态，同时由引用计数保证对象生命周期。

clone 根据标志选择共享地址空间、文件描述符组、信号动作和 TLS；vfork 会阻塞父任务，直到子任务完成 exec 或退出。exec 保留 PID 和部分进程属性，但替换 ELF 映像、重新设置用户栈和入口点，并按 FD_CLOEXEC 关闭描述符；执行前还需要收敛同一线程组内共享执行资源的线程。waitpid 与 waitid 默认按线程组范围查找和唤醒子进程，使创建子进程的线程与回收子进程的线程可以不同。

Rust 的 Arc 和 Weak 对这一设计非常重要。子进程对象可能同时被调度队列、父进程子表和其他内核路径引用，因此需要 Arc 保证对象不会被过早释放；父进程引用则适合使用 Weak，避免父子进程之间形成循环引用。文件对象、Dentry 和超级块等共享资源也通过 Arc 延长生命周期。

在多核路径中，引用计数只解决对象存活问题，不能取代互斥和状态转换。内核在阻塞、上下文切换、关闭 fd、回收子进程或触发 COW 前，会先缩小 TaskLock、文件系统锁和虚拟内存锁的持有范围；可能回调文件对象、访问用户页或唤醒其他任务的操作在相应锁外完成，从而避免锁反转、递归进入和可变借用跨调度点。

2.4 虚拟内存：同时支撑隔离、装载与按需分配

虚拟内存模块解决的是三个相互关联的问题：第一，不同用户进程之间必须相互隔离；第二，ELF 装载、用户栈、堆和 mmap 需要被组织在统一地址空间中；第三，fork 和文件映射不能在一开始就复制或读取所有数据，否则会带来明显的资源浪费。因此，ScintillaOS 以 MemorySet 表示进程地址空间，以 MapArea 表示连续虚拟区间，以页表项表示虚拟页到物理页的映射和权限。

这一路线的核心优势是把“地址区间语义”和“物理页实际分配”分离开来。MapArea 描述某段虚拟地址是否合法、具备什么权限、是否来自文件、是否支持 COW；页表项则描述当前是否已经建立到物理页的映射。这样，缺页异常就不再只是错误，而是内存模块实现按需分配、文件映射和 COW 的触发点。

表 2.1　页错误场景与处理方式

场景	判断依据	处理方式
文件映射首次访问	地址落在合法 `MapArea`，但尚未建立物理页映射	分配页帧；只读、可缓存页面通过文件页缓存读取内容，再建立页表映射
COW 写入	页表当前只读，但区域原本具有写语义，且物理页被共享	分配新页、复制旧页、更新当前页表并恢复写权限；必要时对其他活跃 CPU 执行 TLB 刷新
`MADV_DONTNEED`	VMA 指定可丢弃区间	失效页表项、刷新活跃 CPU 的 TLB，并在安全顺序下回收实际物理页
非法访问	地址不属于用户空间，或访问权限与页表权限不匹配	返回错误、递送异常语义，或终止当前进程

选择这种设计，是因为它能够让 `fork`、`mmap` 和 ELF 装载共享同一套地址空间管理逻辑。fork 不需要专门复制全部物理页，只需要修改页表权限并标记 COW；mmap 不需要在调用时立即读取完整文件，可以在首次访问时按页装入；ELF 装载也可以通过 MapArea 管理代码段、数据段、用户栈和权限。这样既减少了重复代码，也使缺页处理成为内存模块的统一入口。

页表权限是用户态隔离的基础。读、写、执行和用户位不仅决定访问是否成功，也决定异常处理时内核如何解释错误原因。例如，用户程序向只读代码段写入，应被视为权限错误；对 COW 共享页写入，则应被解释为合法的写时复制；访问未映射但属于合法文件映射区域的地址，则应触发按需装页。三者在硬件层面都可能表现为页错误，但在内核语义上完全不同，因此必须结合 `MapArea` 元数据和页表项状态共同判断。

文件型 `execve` 不再预读整个 ELF 映像或预分配完整用户栈。装载器先为 `PT_LOAD` 段建立文件后备或匿名 VMA；完整文件页在首次读、写或执行缺页时装入，只读、可执行页可由全局文件页缓存共享，私有可写页保持进程私有。段首尾非页对齐部分和 BSS 边界在装载时单独处理，避免将无关文件字节暴露给用户态；8 MiB 用户栈只预分配参数、环境变量和 auxv 实际占用的顶部页面，随后按

缺页向下增长。

地址空间维护 active CPU mask。`munmap`、COW 权限变化和共享 VM 元数据修改会先更新当前页表，再对其他正在使用该地址空间的 CPU 发送远端 TLB 刷新请求；旧页帧只有在刷新完成后才会释放，避免其他 CPU 使用陈旧 TLB 条目访问被复用的物理页。文件页缓存仅共享不可写的文件页，`MAP_PRIVATE` 的可写页保持进程私有，从而不破坏 COW 隔离。

Rust 的 RAII 机制适合管理物理页帧生命周期。`FrameTracker` 拥有物理页帧，在生命周期结束时通过 `Drop` 自动归还页帧；共享物理页则配合引用计数延迟释放。这种方式可以减少异常路径上的资源泄漏和重复释放问题。但页表项写入、地址翻译和用户缓冲区访问仍然是内核中最危险的部分，因为它们跨越了 Rust 类型系统与真实硬件状态。ScintillaOS 因此将这些操作集中封装，通过接口约束减少 unsafe 的扩散范围。

2.5 VFS 与文件描述符：用统一抽象连接磁盘、管道、设备和 proc

用户程序并不关心某个 fd 背后是普通文件、目录、管道、设备还是 proc 节点，它只期望通过 `open/read/write/close` 等接口完成访问。因此，文件系统模块需要解决的问题不是单纯读取 Ext4，而是将多种内核对象统一成“可以通过 fd 操作的资源”。这也是 ScintillaOS 引入 VFS、Dentry、挂载表和 `File` trait 的原因。

VFS 的设计目标是把路径解析、文件系统实例和打开文件对象分离。路径解析负责将 `/mnt/testcase` 这样的字符串逐级解析为目录项；Dentry 负责缓存路径分量与底层 inode 或挂载点之间的关系；超级块表示一个具体文件系统实例；`File` trait 则表示已经打开、可以被 fd 表引用的对象。这样，系统调用层不需要知道某个 fd 的底层类型，只需要调用统一的 read、write、stat 或 close 接口。

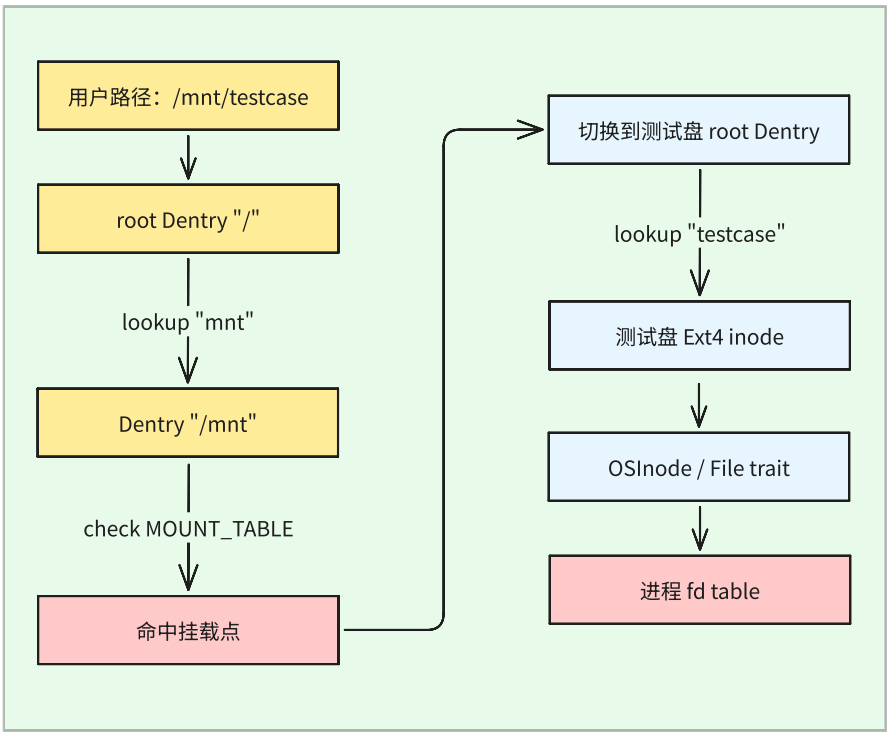


图 2.3　VFS 挂载点穿越流程

选择 VFS 路线的另一个原因，是 ScintillaOS 需要支持双盘和多类特殊文件。根文件系统和测试盘分别对应不同 VirtIO 块设备，`/mnt` 之后的路径应进入测试盘超级块，而不能继续访问根盘。如果仅把路径当作普通字符串拼接，就很难正确处理挂载点穿越、路径缓存和不同文件系统实例之间的切换。

pipe、设备文件、proc 文件、eventfd、epoll、Unix 域 socket 与网络 socket 都实现统一 fd 生命周期。打开文件的最后一个目录链接被删除时，VFS 会将仍被引用的普通文件转为内部 orphan 路径，并在最后一个 `OSInode` 关闭后再实际删除；`ftruncate` 直接作用于活动文件句柄并同步 inode/Dentry 缓存。该语义使临时文件、重命名和覆盖写入能够在复杂构建中保持正确。

普通文件写入先按 `(superblock, inode)` 合并为有序脏区间；`fsync`、`fdatasync`、`sync`、截断、内存压力和关机等可观察边界再触发回写。路径解析使用可失效的正、负 Dentry 缓存和目录 namespace epoch：并发 lookup 只有在前后代际一致时才能发布缓存。创建、删除和重命名则在底层按父目录 inode 重新验证并在同一事务中完成，避免“缓存命中后目录已变化”的 TOCTOU；重命名覆盖的打开文件和硬链接仍保持旧 inode 可见性，直到最后一个引用关闭。

文件系统模块也与进程和内存模块紧密相关。进程的 fd 表决定用户程序能访

问哪些打开文件；fork 需要复制 fd 表并共享底层文件对象；exec 通常保留未设置 close-on-exec 的 fd；mmap 则需要从文件系统读取页面内容并交给内存模块建立映射。因此，VFS 不是孤立的文件访问层，而是连接进程生命周期、地址空间和设备驱动的重要中间层。

多核路径中，VFS、全局 lwext4 状态、单个 Ext4 实例、Dentry 和设备访问按固定锁层级组织。文件系统闭包返回后再更新外层元数据，任务锁内只移动需要关闭的 fd 并在锁外调用关闭回调，避免文件系统、调度器和任务生命周期之间形成锁反转。

2.6 Linux 系统调用兼容：在 ABI 边界上保持语义一致

如果说陷阱入口解决的是“用户如何进入内核”，那么 Linux syscall ABI 兼容解决的就是“用户程序进入内核后期望看到什么行为”。系统调用兼容不能只理解为实现同名函数，它还包括寄存器传参、调用号、返回值符号、错误码、用户指针读取和各种边界条件。任何一个细节不一致，都可能导致 BusyBox、LTP 或比赛测例出现难以定位的异常。

在 RISC-V Linux ABI 中，系统调用号由 `a7` 传入，最多六个参数由 `a0--a5` 传入。内核需要在系统调用分发层完成参数读取，然后把语义处理转交给进程、文件、内存、网络等模块。成功时返回非负值，失败时返回负 errno。对于未实现的系统调用，返回 `-ENOSYS` 比直接伪造成功更安全，因为用户程序可能依赖该错误码进入降级路径，或者让测试明确暴露缺失能力。

ScintillaOS 选择将系统调用层设计为“ABI 解码 + 模块转发”的薄层，而不是把所有逻辑堆在 syscall 文件中。这样做的原因是，系统调用本身只是用户态访问内核能力的入口，真正语义属于对应子模块。例如，`read` 的核心语义属于 fd 和 File trait；`fork` 属于进程和内存模块；`mmap` 同时涉及文件系统和虚拟内存；`waitpid` 则依赖进程父子关系和退出状态。保持这种分层可以让系统调用分发逻辑更清晰。

当前系统调用层还分派 `eventfd2`、`epoll_create1`、`epoll_ctl`、`epoll_pwait`、`socketpair`、`sendmsg`、`recvfrom` 等接口。`clone`、`exec`、`wait`、futex 与信号调用需要同时遵守线程组、共享资源、阻塞唤醒和用户上下文语义；`fcntl` 对终端、管道和普通 fd 的状态修改必须保持同一 open-file-description 的共享语义。系统调用

支持状态不能只由 match 分支是否存在决定。一个 syscall 是否真正可用，取决于其 ABI、主要语义、错误码和关键边界条件是否与测试期望一致。ScintillaOS 在描述支持程度时，需要同时结合源码路径和测试日志，区分“能够返回”“最小兼容”和“完整语义较一致”这几种不同层次。

2.7 并发同步与资源管理

内核内部的任务队列、进程表、文件描述符表、页帧分配器、Dentry 缓存、Ext4 实例和设备队列会被多个 CPU 并发访问。系统使用领域锁而不是单一全局锁保护这些对象：CPU 本地状态、PID 注册表、futex、调度器、共享文件表、任务、信号动作、虚拟内存、页帧引用计数、页帧分配器、VFS、Ext4、Dentry 和设备锁按固定层级获取。锁不得跨越调度、阻塞 I/O、用户地址转换、页表修改、SBI 调用、可能重新进入内核的内存分配或外部回调。

为避免 Rust/Cargo 高并发负载将所有内核元数据分配串行化，8 B 至 4 KiB 的高频小对象优先使用每 CPU 缓存；动态内核堆按 CPU 分片，释放时根据地址归还原分片。小内存配置保持保守的堆增长比例；可用内存不少于 2 GiB 时，动态区可按较大比例增长但设有 960 MiB 上限，8 GiB 配置下内核堆总量约为 1 GiB。同时对全局延迟写缓存设置 64 MiB 硬上限，避免扩展内核堆后写缓存挤占页表、任务和 VFS 元数据。

统一阻塞协议使用 `Running -> Blocking -> Blocked -> Ready` 状态转换。任务先在 SchedulerLock 下发布等待来源和 `Blocking` 状态，再释放调度锁并重新检查对象条件；若条件已满足，当前 CPU 取消阻塞并继续执行；若竞争唤醒发生在上下文保存前，唤醒方只写入 `WakePending`，由拥有该任务的 CPU 在切换完成后发布为 `Ready`。pipe、wait、futex、带截止时间的睡眠以及部分 poll/UDP 路径使用该协议。

调试和 `smp-diagnostics` 构建启用 lockdep。它记录每个 CPU 的持锁栈、锁等级、实例地址和获取位置，检测递归加锁、等级逆序、同等级地址逆序以及切换或 idle 前遗留锁。该机制用于定位设备缓冲区锁顺序、父子任务回收、文件关闭回调、COW 页访问和 Ext4 闭包等跨模块死锁风险。

资源管理方面，ScintillaOS 尽量用 Rust 所有权表达内核对象生命周期。`FrameTracker` 管理物理页帧，`PidHandle` 管理 PID，`Arc<dyn File>` 管理 fd 指

向的打开文件对象，`Arc` 和 `Weak` 管理进程树、Dentry 和超级块之间的引用关系。这样做的原因是，内核错误往往不是单个分支判断错误，而是资源在异常路径中释放过早、释放过晚、重复释放或形成循环引用。

不过，Rust 并不能自动证明整个内核安全。用户指针访问、页表修改、设备 MMIO、DMA 缓冲区、上下文切换和陷阱返回都需要突破普通类型系统。ScintillaOS 的实现原则是：普通业务逻辑尽量使用安全 Rust；必须 unsafe 的地方集中在硬件边界、地址转换和上下文切换附近，并通过接口封装限制其影响范围。这样可以让代码审查和测试更聚焦，避免 unsafe 逻辑在内核中无边界扩散。

2.8 平台与设备适配：在 QEMU virt 上验证完整内核路径

ScintillaOS 面向 QEMU RISC-V virt 与 QEMU LoongArch virt 平台。RISC-V 路径使用 OpenSBI、Sv39、VirtIO MMIO 块设备和网卡；LoongArch 路径适配本架构的启动、陷阱、定时器、IPI、页表，并通过 PCI 枚举、BAR 配置和 `PciTransport` 接入 VirtIO 块设备和网卡。上层的任务、内存、VFS、系统调用和网络状态机在两种架构间复用。

平台适配需要处理启动约定、异常与内存管理、设备与中断三类差异。RISC-V 和 LoongArch 在入口寄存器、页表格式、TrapFrame、IPI 和 VirtIO 传输层不同；这些差异被限制在架构相关代码和驱动边界，避免扩散到进程、文件系统和系统调用语义。LoongArch 网卡不再沿用 RISC-V 的 MMIO 扫描方式，而是从 ECAM 枚举 PCI 设备、为 BAR 分配地址并启用 memory space 与 bus master，再复用 smoltcp interface 和 socket 状态机。

QEMU 不仅用于启动内核，还提供可重复的双盘、多核、虚拟网卡验证环境。构建和测试记录应包含内核 commit、架构、CPU 数、内存大小、根盘与测试盘来源、QEMU 命令、网络参数和 judge 版本。已归档记录覆盖 RISC-V64 与 LoongArch64 的八核 CAgent 和 BuildStorm Q2.1；RISC-V 的 Q2.2 已完成 `tg-xtask` 预构建并进入正式构建与产物启动验证，尚不能据此写成完整通过；四核 RISC-V 回归还覆盖 BusyBox、lmbench、libc-test 与 iperf 等初赛测试。

2.9 小结

本章说明了 ScintillaOS 的核心技术路线：以内核启动和统一陷阱建立用户态与内核态之间的可信边界；以 TCB、线程组、地址空间和 fd 表组织进程资源；以每 CPU 调度、领域锁、统一阻塞唤醒、IPI 和远端 TLB 刷新保证并发正确性；以 COW、文件型 ELF/用户栈按需装载、页缓存、VFS、Ext4 和系统调用分发支撑 Linux 兼容语义；再通过双架构 VirtIO 设备和可重复 QEMU 测试完成端到端验证。各模块围绕同一个目标展开：让用户程序以接近 Linux 的方式使用文件、内存、进程、事件和网络资源，并保持任务隔离、资源生命周期与跨核可见性的正确性。

3 SMP 调度与并发同步设计

ScintillaOS 的 SMP 设计目标不是复现 Linux CFS 一类通用调度器，而是在 QEMU 多核环境中首先建立可验证的任务所有权、阻塞唤醒和跨核可见性语义。系统采用“每 CPU FIFO 就绪队列 + 全局调度器事务锁”的第一代 SMP 架构：队列本身按 CPU 划分以保留局部性，任务取放、状态转换和等待队列登记由同一把 SchedulerLock 串行化。该取舍减少了多队列无锁迁移的状态组合，优先避免同一任务被两个 CPU 同时恢复、已经阻塞的任务丢失唤醒、以及旧页表项在其他 CPU 上继续可见等错误。

当前内核的调度器最多容纳 12 个 CPU。RISC-V 平台默认最多使用 8 个应用 CPU，LoongArch 平台默认最多使用 12 个应用 CPU；实际参与调度的 CPU 由启动过程中动态发布的在线掩码决定，因此同一内核也可在较小的 `-smp` 配置下运行。

3.1 设计目标与核心不变量

SMP 化后，单处理器中“当前任务”和“任务状态”天然一致的前提不再成立。ScintillaOS 将正确性目标拆分为三个可独立检查的不变量：第一，每个任务至多拥有一个就绪队列条目；第二，每个任务同时至多由一个 CPU 执行；第三，阻塞任务在真正离开 CPU 前不能被其他 CPU 直接重新放入就绪队列。前两个不变量由 TCB 中的原子元数据维护，第三个不变量由调度状态机和 idle 上下文中的延迟发布共同维护。

表 3.1 SMP 调度的核心对象与所有权

对象	所有者或保护方式	作用
`CpuLocal`	当前 CPU 的 per-CPU 存储与本地中断屏蔽	保存当前任务、idle 上下文、时钟 tick 和用户浮点状态所有者，不允许远程 CPU 直接访问。
每 CPU FIFO 就绪队列	SchedulerLock	保存可运行任务；普通入队位于队尾，唤醒或优先提升位于队首。
`running_cpu`	原子 compare-exchange	记录正在执行该任务的 CPU；空闲值为 `usize::MAX`。
`enqueued`	原子 compare-exchange	记录该任务是否已经拥有一个就绪队列条目，重复入队或无条目出队会触发断言。
在线 CPU 掩码	原子位图	仅包含已经完成本地初始化并可接收调度工作的 CPU，防止向不存在或尚未就绪的 hart 投递任务。
`PENDING_IPI`	每 CPU 原子位图	保存远程 CPU 请求的重调度或 TLB 刷新工作；硬件 IPI 只负责通知，不承载任务状态。

TCB 的可调度元数据与普通可变进程状态分离。`running_cpu`、`last_cpu`、`allowed_cpus` 与 `enqueued` 使用原子操作，使调度器能够在不长时间持有 TaskLock 的情况下确认任务归属；文件描述符表、地址空间、信号状态和父子关系等较大对象仍由领域锁保护。这样的划分并不把原子变量当作第二套锁，而是让它们表达队列成员资格和 CPU 所有权这两个跨模块可观察的事实。

3.2 CPU 本地状态与多核启动协议

每个 hart 在 polyhal 的 per-CPU 区域内拥有独立的 `CpuLocal`。其中包含当前 TCB 引用、从用户任务返回时落脚的 idle 上下文、时钟计数器和浮点状态标识。访问本地状态时，CpuLocalCell 会临时关闭本地中断并检查是否递归借用；它只保护当前 CPU 的私有数据，不作为跨 CPU 互斥手段。内核还将启动协议中的稳定 hart ID 与 per-CPU 基址建立一次映射，避免依赖次级 CPU 上不稳定的通用 hart 查询结果。

多核启动分为两个发布阶段。次级 hart 首先只初始化自己的 per-CPU 存

储和 idle 上下文，然后以 release 语义设置 SMP_PERCPU_READY 的对应位，并等待 CPU0 完成全局初始化。CPU0 在导入剩余物理内存之前等待所有配置 CPU 的 per-CPU 区域就绪，避免将尚未使用的 per-CPU 存储误交给帧分配器。随后 CPU0 初始化内核堆、页帧分配器、页表、文件系统、设备和初始任务，并发布 SMP_GLOBAL_INIT_DONE。

次级 hart 即使观察到全局初始化完成，也不会立即加入调度器。它继续等待 CPU0 完成首次任务交接，即第一个用户任务已拥有有效的 CPU 归属和用户页表，之后才通过 register_online_cpu() 设置在线位并进入本地 run_tasks() 循环。该顺序避免空闲 CPU 在第一个任务的上下文、页表或 SchedulerLock 状态尚未稳定时接收 IPI 或尝试偷取任务。

3.3 每 CPU 就绪队列、亲和性与任务选择

调度器在 SchedulerLock 内维护一个固定长度的 FIFO 队列数组，同时记录各 CPU 当前正在运行的 PID。新任务入队时先取任务允许 CPU 掩码与在线 CPU 掩码的交集；在候选 CPU 中选择“就绪队列长度加当前运行任务数”最小的 CPU。若多个候选 CPU 负载相同，优先选择任务的 last_cpu，从而保留缓存局部性并减少不必要迁移。普通入队使用队尾，因事件唤醒或优先提升而重新可运行的任务使用队首。

每个 CPU 优先从本地队列队首获取任务。当本地队列为空时，调度器顺序检查其他 CPU 的队列，从中寻找允许在当前 CPU 上运行的第一个任务并移除。偷取时若被检查 CPU 没有正在运行任务，则保留其队列中的一个任务，避免一个刚空闲的 CPU 在下一轮调度时没有可运行工作；因此工作偷取用于缓解明显不均衡，而不是每个时间片都强制迁移任务。

任务亲和性更新也在 SchedulerLock 事务中完成。若任务处于 Ready 状态，调度器先从旧队列移除其唯一条目，再按新掩码重新选择目标 CPU；若任务正在运行，本实现不抢占式迁移其上下文，而是在下一次让出、阻塞或时钟抢占后依据新掩码重新入队。这样避免在另一个 CPU 正在保存寄存器现场时移动任务所有权。

表 3.2　就绪队列操作及其调度语义

操作	队列位置	语义
创建任务或普通让出	队尾	根据允许 CPU、在线 CPU、队列长度和 `last_cpu` 选择目标，保持 FIFO 公平性和局部性。
事件唤醒、超时到期、优先提升	队首	尽快重新检查已满足的对象条件，降低 pipe、wait、futex、信号等路径的唤醒延迟。
本地取任务	本地队首	CPU 优先运行本地工作，减少跨 CPU 迁移。
空闲偷取	其他队列的可运行条目	仅在本地无任务时执行，遵守任务亲和性，并为无运行任务的被偷取 CPU 保留一个条目。
修改亲和性	删除后重新入队	只迁移 Ready 任务；Running 任务在下一个安全调度边界生效。

这一设计的关键点是“每 CPU 队列”并不意味着无锁并发访问。所有队列变更仍通过全局 SchedulerLock 串行化，因而可以在一个线性化点内同时维护队列成员资格、运行记账和任务状态。系统以此换取简单的正确性证明：就绪队列负载均衡是近似策略，任务唯一所有权才是必须保持的语义。

3.4　任务状态机与上下文交接

任务状态包含 Ready、Running、Switching、Blocking、Blocked、WakePending 和 Zombie。Ready 表示任务恰有一个就绪队列条目；Running 表示该任务已被一个 CPU 取走并由 `running_cpu` 标识；Switching 表示任务主动让出或被时钟抢占，寄存器现场尚在保存，不能提前再次执行。Blocking、Blocked 和 WakePending 用于处理阻塞与并发唤醒，Zombie 则表示任务已经退出并等待回收。

`fetch_task()` 在 SchedulerLock 和 TaskLock 的固定嵌套顺序下完成 Ready 到 Running 的转换：先从队列移除条目并清除 `enqueued`，再检查任务仍为 Ready，随后设置 TaskStatus 为 Running、登记调度器中的运行 PID，并通过 compare-exchange 发布 `running_cpu`。只有全部步骤完成后，选中的 TCB 才会交给具体 CPU 的调度循环。因此，另一个 CPU 不会观察到“仍在队列中但已运行”或“已

运行但没有所有者”的中间状态。

从用户任务返回 idle 上下文时，`run_tasks()` 才调用 `complete_context_switch()` 完成离开任务的状态转换。任务在发起 `schedule()` 前仍被保留在本 CPU 的 `CpuLocal.current` 中，且本地中断被关闭；只有底层上下文切换已经将寄存器保存到任务的内核上下文、CPU 已回到 idle 栈后，`take_current_task()` 才清除 `running_cpu`，并由 `complete_context_switch()` 发布新的状态。这个延迟发布窗口是防止另一 CPU 恢复半保存上下文的关键。

时钟中断只在用户态返回路径上触发抢占，内核态代码保持非抢占，以免在持锁或使用本地内核栈时发生异 CPU 恢复。RISC-V 的调度时钟按一个 tick 判断时间片；LoongArch 保留 1 ms 的硬件定时精度用于睡眠和定时器，但每 10 个 tick 才形成一个用户调度时间片。若当前 CPU 的本地队列没有其他可运行任务且亲和性没有改变，内核会跳过无意义的周期性任务切换；一旦存在本地工作、亲和性变化或显式唤醒请求，下一安全边界即可重新调度。

3.5 统一阻塞、二次检查与并发唤醒

阻塞操作面临的典型竞态是：任务先检查“对象不可用”，准备睡眠时另一个 CPU 已经使对象可用并发送唤醒；若等待者尚未登记到等待队列，唤醒会永久丢失。ScintillaOS 对 pipe、网络 socket、waitpid、带截止时间的睡眠、文件页装载等待和部分信号路径使用统一的两阶段协议。

第一阶段，任务在 SchedulerLock 下从 Running 变为 Blocking，同时记录 `BlockReason` 并加入相应等待队列，例如网络等待队列、waitpid 队列或睡眠截止时间表。第二阶段，释放调度锁后重新检查对象条件；如果对象已经可用，`cancel_block_before_switch()` 会删除等待登记并将 Blocking 或 WakePending 恢复为 Running，当前 CPU 继续执行。只有二次检查仍然失败时，任务才真正切换到 idle 上下文。

并发唤醒者同样在 SchedulerLock 中检查任务状态。若任务仍是 Blocking，说明其上下文尚未完全保存，唤醒者只把状态改为 WakePending，而不向任何就绪队列添加条目；若任务已经是 Blocked，则把它改为 Ready、放入目标队列队首，并在释放锁后向目标 CPU 发送重调度请求。最终，拥有该任务的 CPU 在 idle 上下文中看到 WakePending，确认寄存器现场已安全保存后才把任务发布为 Ready。

这样，“检查条件、登记等待者、二次检查、上下文保存和队列发布”不会留下可被其他 CPU 观察到的丢失唤醒窗口。

信号发送还涉及另一种 Ready 到 Running 的竞态。发送者发现目标为 Ready 后，目标可能已经被其他 CPU 从队列取走。`add_priority_task()` 因此在 SchedulerLock 后再持有 TaskLock 复核状态：若仍为 Ready，只提升既有队列条目；若已经 Running，则不创建第二个条目，而是请求 `running_cpu` 尽快重新调度并观察 pending signal。这一规则与 `enqueued`、`running_cpu` 断言共同防止重复入队。

3.6 IPI、远端 TLB 刷新与跨核可见性

调度器发布新的就绪任务或高优先级唤醒后，不会在持有 SchedulerLock 时直接执行 SBI 或设备 IPI 调用。它先完成队列和任务状态的原子发布，释放所有领域锁，再由 `request_reschedule()` 向目标 CPU 设置 `PENDING_IPI` 的重调度位并发送硬件通知。RISC-V 使用 OpenSBI 的 IPI 接口；LoongArch 先以顺序一致栅栏发布邮箱，再向 IPI 设备发送通知。接收 CPU 在软件中断处理程序中确认硬件中断，以 acquire-release 语义取走 pending 位；该处理程序只让 CPU 从 `wfi` 或 idle 中返回以重新检查本地队列，不在中断上下文直接切换任务。

共享地址空间还需要保证页表修改对所有活跃 CPU 可见。每个地址空间维护 active CPU mask；`munmap`、COW 权限变化、共享 VMA 变更和 `MADV_DONTNEED` 会先修改当前页表，再向仍在使用该地址空间的远程 CPU 发起 TLB 刷新，旧帧必须在刷新完成后才允许回收。RISC-V 使用 OpenSBI remote `sfence.vma`；LoongArch 为每个目标 CPU 发布递增的刷新序列号，远程 CPU 执行本地全 TLB 刷新后写入确认序列号，发送者只等待仍处于 active mask 中且尚未确认的 CPU。若目标 CPU 在请求期间退出该地址空间，它已切回内核页表，下一次进入用户地址空间前会刷新本地 TLB，因此可作为完成该请求的静默点。

3.7 领域锁、锁顺序与诊断机制

SMP 内核中的锁不仅需要保护单个对象，还必须约束跨模块调用链。ScintillaOS 为 CPU 本地状态、PID 注册表、futex、调度器、共享 fd 表、TCB、信号动作、虚拟内存、VFS、页缓存、页帧、Ext4、Dentry 和设备队列定义领域锁，并以固定等级限制嵌套顺序。调度相关的关键规则是 SchedulerLock 可以在需要时

再获取 TaskLock，而不允许反向持有 TaskLock 后进入调度器。

表 3.3 领域锁等级与主要保护对象

等级	锁类别	主要对象
0–3	CpuLocal、PidRegistry、Futex、Scheduler	本地当前任务、PID 到 TCB 映射、futex 等待表、就绪/等待/回收队列。
4–6	Files、Task、SignalActions	共享 fd 表、单个任务可变状态、线程组共享信号处置。
7–9	Vm、VmPages、VmPageTable	地址空间拓扑、VMA 常驻页、页表遍历与 PTE 更新。
10–11	Vfs、PageCache	挂载与路径元数据、只读文件页缓存。
12–13	FrameRef、FrameAllocator	共享页引用计数和物理页 buddy 分配器。
14–18	Fs、FsGlobal、Dentry、FsAux、Device	Ext4 实例、全局 lwext4 注册表、目录缓存、文件系统辅助状态和设备或管道环形缓冲区。

领域锁均为非可重入锁。内核不允许持锁跨越调度、阻塞 I/O、用户地址翻译、页表修改、SBI 调用、可能重新进入内核的内存分配或外部回调。典型处理方式是：在 TaskLock 内仅移动需要关闭的 fd，释放锁后再调用文件对象的关闭回调；在 Ext4 闭包返回后再更新 VFS 元数据；在发送 IPI 前释放 SchedulerLock。这样既避免等级反转，也避免持锁路径因缺页、COW、设备完成或回调而重入调度器。

`smp-diagnostics` 构建启用 lockdep。它按 CPU 记录持锁栈、锁类别、实例地址、静态名称以及获取位置，检查递归加锁、锁等级逆序和同等级多实例的地址逆序；在进入 `schedule()`、阻塞或 idle 等边界前还会断言没有遗留自旋锁。`lock-stats` 进一步记录调度循环、任务交接、idle 等待、定时 tick、锁竞争和等待时间。发生停滞时，诊断快照可同时给出各 CPU 的调度阶段、当前 TID/TGID、TCB 地址、pending IPI、队列长度、阻塞原因和持锁栈，从而把“多核卡死”缩小到可定位的状态转换或锁顺序问题。

3.8 设计边界与小结

当前实现把正确性置于调度策略之前。每 CPU FIFO 队列、局部优先、空闲偷取和亲和性约束足以支持多核 QEMU 中的并行用户任务、线程组、阻塞 I/O 和复杂构建负载，但全局 SchedulerLock 仍是就绪队列操作的线性化点，系统尚未实现 Linux CFS、NUMA 感知调度、强制运行中迁移、实时调度类别或完整的负载均衡器。CPU0 还承担网络轮询、全局睡眠队列和部分定时器检查，以降低中断侧跨锁交互复杂度。

本章的实现路线可以概括为：以 per-CPU 状态隔离本地执行上下文；以 SchedulerLock、`enqueued` 和 `running_cpu` 保证任务唯一所有权；以 Blocking 和 WakePending 将阻塞唤醒延迟到上下文安全保存后发布；以 IPI 和远端 TLB 刷新建立跨 CPU 的工作通知与页表可见性；再以领域锁、lockdep 和运行时快照约束跨模块并发。该设计为后续进程、内存、文件系统和网络模块在多核环境中共享资源提供了统一的同步基础。

4 内存管理模块

4.1 虚拟内存机制

内存管理首先要解决的是地址空间隔离问题。ScintillaOS 面向 RISC-V 平台时采用 Sv39 三级页表，虚拟页号被拆分为三级索引，页表项记录物理页号以及读、写、执行、用户访问等权限。选择这一路线的原因在于，Linux 用户程序默认相信自己运行在独立地址空间中；如果缺少页表隔离，进程之间的内存、用户态与内核态之间的边界都会失去硬件约束。

`MemorySet` 表示一个进程拥有的完整地址空间，内部保存页表和若干 `MapArea`。`MapArea` 描述连续虚拟区间的用途、权限、映射类型、已分配页帧以及可能的文件后备。这样设计的好处是把“某段地址是否合法”与“物理页是否已经分配”分开：ELF 段、用户栈、堆、匿名映射和文件映射都可以通过同一套区域操作完成建立、删除和拆分，缺页处理也能根据区域元数据判断是按需分配、COW 复制还是非法访问。

当前 `MemorySet` 将页表、VMA 元数据、地址空间活动 CPU 掩码和 `brk` 值分别组织为共享对象。普通 `fork` 创建新的页表并复制映射元数据；`clone(CLONE_VM)` 则共享页表、`VmRwLock<Vec<MapArea>>`、活动掩码和程序断点，使同一线程组中的任务始终观察同一份地址空间拓扑。RISC-V 使用 Sv39，LoongArch 在架构相关页表实现中完成同等的用户映射、权限和本地 TLB 切换；上层的 `MemorySet`、VMA 和缺页语义在两种架构间复用。

地址空间不再只记录“是否有页表”，还记录哪些 CPU 当前可能通过该页表访问用户页。调度器在切入用户页表后调用 `enter_cpu` 设置活动位，在切回内核页表后调用 `leave_cpu` 清除活动位。页表变更据此精确确定远端 TLB 刷新目标，避免将已经离开该地址空间的 CPU 纳入等待，也避免漏刷仍在运行共享地址空间的 CPU。

4.2 物理内存分配

地址空间管理最终必须落到物理页帧上。ScintillaOS 由 `frame_allocator.rs` 管理物理页帧，并使用伙伴系统分配器完成页帧申请与回收。选择伙伴分配的原因是页帧天然以固定大小对齐，分裂与合并逻辑清晰，适合内核早期建立页表、用户栈、COW 页面和文件缓存页面时反复申请整页内存。内核堆在启动阶段初始化，默认配置为 128 MiB，用于承载 `Arc` 对象、文件系统缓存和各类控制结构。

页帧不能只用裸物理地址表示，因为裸地址无法表达“谁负责释放”。ScintillaOS 使用 `FrameTracker` 以 RAII 方式封装页帧：对象创建时清空页面，复制共享页时增加引用计数，生命周期结束时自动归还或递减引用。代码4.1摘自 `os/src/mm/frame_allocator.rs:25--61`，展示了这一资源边界。

代码 4.1　`FrameTracker` 的构造、共享与释放

```
pub struct FrameTracker {
    pub paddr: PhysAddr, // 被跟踪物理页的起始地址
}
impl FrameTracker {
    pub fn new(paddr: PhysAddr) -> Self {
        paddr.clear_len(PAGE_SIZE); // 分配后先清空整页
        Self { paddr }
    }
    pub fn shared_clone(&self) -> Self {
        ref_inc(self.paddr); // 增加共享页引用计数
        Self { paddr: self.paddr }
    }
    pub fn is_shared(&self) -> bool {
        let refs = FRAME_REFS.lock();
        refs.get(&ref_key(self.paddr)).map_or(false, |&c| c > 1)
    }
}
impl Drop for FrameTracker {
    fn drop(&mut self) {
        let mut refs = FRAME_REFS.lock();
        let key = ref_key(self.paddr);
        if let Some(count) = refs.get_mut(&key) {
            // 仍有共享引用时只减少计数，不归还物理页
            if *count > 2 { *count -= 1; } else { refs.remove(&key); }
            return;
        }
```

```
27         drop(refs);
28         frame_dealloc(self.paddr); // 最后一个引用释放物理页
29     }
30 }
```

页帧引用计数当前采用原子计数，而不是单一全局引用表。`FrameTracker::shared_clone()` 只增加引用计数，不复制页面；`Drop` 在最后一个引用离开时才把页交回伙伴分配器。页帧分配器按平台 CPU 数把启动时发现的物理内存区间分入多个 buddy 分片，并为每个 CPU 设置容量为 64 页的本地 magazine；常规单页分配优先命中本地 magazine，未命中时以最多 32 页为一批从本地优先的分片补充，只有本地分片不足时才尝试其他分片。批量文件读取使用独立页批量分配，避免长时间占用一个全局 buddy 锁。

内核堆采用“静态启动堆 + 动态扩展区”的容量策略。启动堆固定为 64 MiB；CPU0 在发现物理内存后预留动态扩展区，小于 2 GiB 的配置按可用内存的 `1/32` 增长，较大内存配置按 `1/8` 增长且最高增加 960 MiB。动态区同样按 CPU 分片，8 B 至 4 KiB 的高频小对象优先从每 CPU 缓存分配，每个 size class 最多保留 64 个空闲对象。该策略在 8 GiB 评测配置下可形成约 1 GiB 的内核堆，同时保持小内存 QEMU 配置的保守占用。

表 4.1　物理页帧与内核堆的当前分层

层次	对象或算法	说明
页帧生命周期	`FrameTracker` 与原子引用计数	分配时清零；共享时增加引用；最后一个 tracker 析构后才归还物理页。
页帧快速路径	每 CPU frame magazine	每 CPU 最多缓存 64 个页号，按批次补充，降低高并发缺页和页缓存装载时的 buddy 锁竞争。
页帧后备路径	分片 buddy allocator	物理内存按平台 CPU 数分片；优先本地分片，不足时回退其他分片。
内核小对象	每 CPU size class 缓存	覆盖 8 B 至 4 KiB 的二次幂规格，减少 TCB、Dentry、VMA 等短生命周期元数据的堆锁访问。
内核大对象与扩展区	启动堆和动态堆分片	64 MiB 启动堆负责早期分配；启动后按内存容量预留分片扩展区，并按地址归还原分片。

4.3 ELF 装载与用户栈

ELF 装载解决的是“磁盘文件如何变成用户进程”的问题。原生 rCore 在 glibc 版本测试不能通过。我们思考发现，ScintillaOS 不能把用户程序简单复制到某个固定地址执行，而必须根据 ELF 程序头建立带权限的地址空间。`MemorySet::from_elf` 解析 `PT_LOAD` 段，为代码、只读数据、可写数据和 BSS 建立映射，并构造用户栈。对于未页对齐的 ELF 段，装载过程保留段首页偏移，保证数据写入正确的虚拟地址；这对 musl 静态程序尤其重要，因为其装载段不一定天然按页对齐。

装载结果不仅包括页表映射，还包括入口地址、用户栈顶、程序头地址等启动信息，这些信息会交给进程创建逻辑写入 TrapFrame。典型用户地址空间由代码段、只读数据段、可写数据/BSS 段、用户栈、堆和 mmap 区域组成。用户栈默认大小为 256 KiB；用户运行时在 `_start` 阶段初始化堆，再解析参数并进入 `main`。因此，ELF 装载同时依赖文件系统、页表和进程上下文，是内存模块连接用户态程序的关键入口。

嵌入内核镜像的初始用户程序仍可使用内存中的 `from_elf` 路径；普通 `execve` 则使用 `from_elf_file`，只读取 ELF 头和程序头表，再为 `PT_LOAD` 建立文件后备的惰性 VMA。完整文件页在首次读、写或取指缺页时才从文件装入。段首非页对齐部分、文件数据尾页以及 BSS 边界由装载器分别处理：首尾页只复制属于当前段的字节并补零，后续 BSS 页作为匿名按需页建立，避免泄露同一文件中相邻段的内容。动态解释器、TLS、PIE 重定位基址、程序头信息和辅助向量均由装载结果传递给用户态启动栈。

当前用户主栈上限为 8 MiB，而不是一次性分配完整栈区。文件型 `execve` 先建立匿名栈 VMA，只预分配容纳 argv、envp、随机数、平台字符串和 auxv 所需的顶部页面；后续向下访问由匿名写缺页逐页扩展。信号返回 trampoline 被映射到独立固定页面，避免正常栈增长覆盖信号恢复代码。该设计减少了 rustc、链接器和 build script 大量启动时的页帧申请与页表操作，同时保持栈上限和用户权限不变。

4.4 缺页、按需分页与写时复制

缺页异常在 ScintillaOS 中不只是错误路径，也是延迟分配资源的触发点。我们在测试 `lmbench` 测试用例时发现，如果按照 rCore 传统 fork 逐页复制父进程，会使创建成本随地址空间线性增长，而许多子进程随后会立即 `exec` 或只读取继承页面。因此 ScintillaOS 将 COW 作为 fork 的默认机制：父子进程最初共享只读物理页，只有某一方写入时才通过写缺页复制页面并恢复写权限。文件映射也采用类似思路，首次访问时才申请页帧并从文件填充数据。

ScintillaOS 通过 `mmap`、`munmap` 和 `msync` 分别负责建立映射、解除映射和脏页回写。这里需要区分两种“共享”：COW 的共享只是复制前的临时状态，写入时应拆分为私有页；`MAP_SHARED` 的共享映射则允许多个映射观察同一物理页或同一文件后备。代码4.2根据 `os/src/mm/memory_set.rs:516--561` 整理了复制用户地址空间的关键路径。

代码 4.2　从父地址空间创建子地址空间的 COW 路径

```
pub fn from_existed_user(user_space: &MemorySet) -> Option<MemorySet> {
    let mut memory_set = Self::new_bare();
    for area in user_space.areas.iter() {
        let mut new_area = MapArea::from_another(area); // 复制映射元数据
        for (&vpn, frame) in area.data_frames.iter() {
            let mut child_perm = area.map_perm;
            let new_frame = {
                let new_frame = frame.shared_clone(); // 父子先共享同一物理帧
                if !area.shared && child_perm.contains(MapPermission::W) {
                    child_perm.remove(MapPermission::W); // 首次写入时再触发 COW
                    user_space.page_table.unmap_page(vpn);
                    user_space.page_table.map_page(
                        vpn, frame.paddr, child_perm.into(),
                        MappingSize::Page4KB);
                }
                new_frame
            };
            // 在子页表中建立同一物理页的只读映射
            memory_set.page_table.map_page(
                vpn, new_frame.paddr, child_perm.into(),
                MappingSize::Page4KB);
```

```
22          }
23      }
24      Some(memory_set)
25  }
```

这段逻辑先以 MapArea 为粒度复制映射元数据，再以虚拟页为粒度处理已经存在的物理帧。对非共享且原本可写的页面，内核移除写权限，并让父子双方引用同一个 FrameTracker。只有写缺页发生时，缺页处理才分配新帧并复制内容。这样，fork 的主要成本从“复制整个地址空间”变为“复制页表元数据和后续真正写入的页面”。

当前缺页处理采用“准备页帧、锁内复核、提交映射”的两段方式。对于文件页，内核可在 VMA 锁外准备读入结果，再取得地址空间和常驻页锁复核 VMA、文件对象、页偏移与代际是否仍匹配；若并发 munmap、mremap 或文件失效改变了元数据，则丢弃准备结果并重新检查。对于私有 COW 写缺页，内核先取得源页的共享引用并在锁外分配、复制，再确认页表和 VMA 未变化后替换为可写私有页。这避免在持有 VmLock、页表锁或文件系统锁时进行可能缺页、分配或 I/O 的操作。

因此，COW 与 mmap 的正确性必须同时观察父子写入隔离、共享页引用计数、文件映射首次访问和解除映射后的资源回收。它们表面上都由缺页异常触发，但背后依赖的语义不同；只有把 MapArea 元数据、页表权限和页帧生命周期放在一起判断，才能避免把合法缺页误处理成进程错误，或把非法访问错误地当成按需分配。

4.5 共享地址空间与堆管理

fork 与 clone(CLONE_VM) 的区别，体现了 ScintillaOS 对地址空间所有权的不同处理。传统 fork 创建新的页表和映射元数据，并通过 COW 延迟实际页面复制；clone(CLONE_VM) 更接近线程式任务，子任务共享父任务的页表 Arc，因此退出时不能提前回收仍被其他任务使用的页帧。这个差异决定了进程管理模块在创建、退出和回收时必须询问内存模块的共享状态。

brk 和 mmap 则分别对应两种用户态内存增长方式。brk 维护连续堆区末端，适合传统堆分配器；mmap 建立离散的匿名或文件映射，适合大块内存、文件映射

和特殊运行时需求。munmap 需要处理完整删除、头部裁剪、尾部裁剪和中间分裂等情况，因此边界判断比普通释放页帧更复杂。ScintillaOS 将这些操作统一落到 MapArea 的区间维护上，是为了让堆、文件映射和缺页处理共享同一套地址空间语义。

共享地址空间中的 VMA 列表和 program_brk 是唯一的规范状态，而不是由各 TCB 维护最终一致的副本。CLONE_VM 成员退出时只分离自己持有的 MemorySet 句柄；只有最后一个地址空间所有者退出后，页表和 VMA 才进入统一回收流程。对于 munmap，内核将从 VMA 中拆下的帧、需要回写的共享文件页和已退休的区域封装在 UnmapResult 中，调用者在锁外完成远端 TLB 刷新后再调用 release()。这种“先失效地址转换、后释放物理资源”的顺序避免其他 CPU 使用陈旧 TLB 访问已经被分配器复用的页面。

用户地址空间的元数据修改由 VmRwLock、常驻页分片锁和页表读写锁分层保护；所有用户指针翻译、文件 I/O、远端 TLB 刷新和最终帧回收均安排在相应锁域之外或之后执行。内核堆的分片策略与用户页帧分配器相互独立：前者服务 Rust 内核对象和文件系统元数据，后者服务页表、匿名页和文件页，因此复杂构建中的 VFS 缓存膨胀不会直接改变用户 VMA 的语义边界。

5 进程管理模块

5.1 核心数据结构

进程管理要解决的是“一个正在运行的用户程序在内核中如何被表示”。与其他操作系统相同，ScintillaOS 采用 `TaskControlBlockInner`(TCB)。这样设计的原因在于，Linux 进程语义本身就是跨资源的：`fork` 需要同时复制地址空间和 fd 表，`exec` 需要替换地址空间但保留部分 fd，`waitpid` 又需要读取退出状态并回收子进程对象。

在此基础上，`TASK_MANAGER` 维护 FIFO 就绪队列，`PID2TCB` 负责按 PID 查找进程控制块。调度器并不直接理解文件系统或内存细节，而是通过 TCB 拿到可以切换的任务对象；系统调用层也不直接保存进程资源，而是通过当前 TCB 访问地址空间、cwd、root 和 fd 表。也就是说，TCB 是进程生命周期的中心节点，其他模块围绕它建立引用关系。

字段类别	作用
TrapFrame 与任务上下文	保存用户寄存器、内核栈指针和恢复执行所需上下文。
`memory_set`	指向任务虚拟地址空间；`CLONE_VM` 任务可共享该对象。
`fd_table`	文件描述符到 `File` trait 对象的映射，支持复制和 close-on-exec 处理。
`cwd/root`	当前目录和进程根目录的 Dentry 引用，用于相对路径和路径隔离语义。
父子关系	父进程弱引用及子进程表；退出时用于收养、wait 和 SIGCHLD。
信号状态	待处理位图、阻塞掩码、信号动作表、冻结/终止标志。
凭证与时间	uid/euid/gid/egid、用户态和内核态累计时间。

字段类别	作用
线程组与调度元数据	TCB 的稳定调度部分记录线程组号、运行 CPU、上次运行 CPU、允许 CPU 掩码、是否已入队、可等待僵尸标志和 vfork 父任务标识；这些字段采用原子访问，避免把调度器状态长期置于资源锁内。
files_id 与 fd 组	每个任务均有文件描述符表组标识。普通 fork 复制表项并拥有新的组；CLONE_FILES 成员共用同一组，由专用文件表锁串行化 close、dup、fcntl 和 exec 时的 FD_CLOEXEC 处理。

代码5.1给出了 fork 路径的关键摘录。该函数刻意分两个阶段读取父任务状态：先复制地址空间，再短时间借用父 TCB 收集其余状态，避免长时间持有内部可变借用。

代码 5.1　TaskControlBlock::fork 的关键资源复制逻辑

```
1  pub fn fork(self: &Arc<TaskControlBlock>) -> Option<Arc<TaskControlBlock
       >> {
2      let memory_set = {
3          let parent = self.inner_exclusive_access();
4          MemorySet::from_existed_user(&parent.memory_set)?
5      };
6      let (trap_cx, fd_table, cwd, root, signal_mask, actions) = {
7          let parent = self.inner_exclusive_access();
8          (parent.trap_cx.clone(),
9           parent.fd_table.iter().map(|f| f.as_ref().map(|x| {
10              x.dup_ref(); x.clone()
11          })).collect(),
12          parent.cwd.clone(), parent.root.clone(),
13          parent.signal_mask, parent.signal_actions.clone())
14     };
15     // 分配新的 PID 和内核栈，再构造子任务控制块。
16 }
```

第 3–5 行调用 MemorySet::from_existed_user 建立子地址空间，这里进入的是 COW 路径，而不是立即复制所有用户页。第 7–14 行复制用户现场、当前工作目录、根目录、信号掩码和动作表。文件描述符对象以 Arc 共享，因此复制 fd

表时增加底层对象引用，而不是复制文件内容。随后子任务分配新的 PID 和内核栈，并把子任务返回值置为 0；父任务则从系统调用返回子 PID。这个差异正是 Linux fork 语义中父子执行路径分叉的来源。

为适应多核调度，TCB 将“调度元数据”和“可变进程资源”分离。`TaskLock<TaskControlBlockInner>` 保护地址空间、fd、信号动作、父子关系等复合对象；运行 CPU、允许 CPU 掩码、线程组号和入队状态则通过原子字段发布。查 PID 的 `PID2TCB` 注册表仅在取得 `Arc` 引用时短暂持锁，后续内存、文件或信号操作不携带注册表锁，从而避免进程查找成为跨模块锁依赖的中心。

5.2 PID 管理与调度器

PID 是用户态观察进程的稳定身份，因此它的分配和释放必须与进程对象生命周期绑定。ScintillaOS 使用 `PidHandle` 管理 PID，在对象释放时自动归还编号，避免异常退出路径遗漏回收。`PID2TCB` 提供从 PID 到 TCB 的索引，供 `kill`、`waitpid`、`tgkill` 和父子关系维护等路径使用。

调度器已由单一 FIFO 队列扩展为每 CPU FIFO 就绪队列。`add_task` 根据任务的允许 CPU 掩码、上次运行 CPU 与队列负载选择目标 CPU；任务的 `enqueued` 标志保证同一个就绪任务不会被重复插入。当前运行任务记录 `running_cpu`，因此 `sched_setaffinity` 修改亲和性时，可以把尚未运行的任务从旧队列移除后重新入队，而不会强制迁移正在用户态或内核态执行的任务。详细的选核、IPI 唤醒与状态交接规则见第三章。

任务可能因 `nanosleep`、网络 I/O、管道、`waitpid` 或 futex 等原因离开就绪队列；事件发生后，内核再将其转为可运行状态。阻塞前必须释放 TCB、文件系统或网络对象上的高层锁，否则任务切换后再次进入相关路径时会形成锁顺序反转。当前阻塞路径使用 `Blocking`、`Blocked` 和 `WakePending` 等中间状态实现“发布阻塞意图、再次检查条件、再切出”的协议，使唤醒方即使与阻塞方并发，也不会把一次有效唤醒丢失。

5.3 创建、切换与执行

进程创建和调度的主线可以概括为如下过程：

fork/clone 创建 TCB → 分配 PID、内核栈和 TrapFrame →
复制或共享地址空间与 fd 表 → add_task 入就绪队列 →
fetch_task 取队首任务 → 切换页表和任务上下文 → 返回用户态

fork、clone 和 execve 都与新执行流有关，但它们的资源语义并不相同。fork 创建新的进程身份和独立页表，并通过 COW 延迟复制物理页；clone 根据标志选择是否共享地址空间、文件描述符表、信号动作或其他资源；execve 不创建新 PID，而是在原进程身份下用新的 ELF 映像替换地址空间和 TrapFrame，并保留未设置 FD_CLOEXEC 的文件描述符。

clone 路径按照标志组合建立资源边界。CLONE_VM 共享地址空间句柄，CLONE_FILES 共享文件描述符表组，CLONE_SIGHAND 共享信号动作；线程式创建还发布共同的线程组号，并按 CLONE_SETTLS 设置新任务的 TLS 上下文。普通 fork 则复制文件描述符表项、继承 cwd、root、凭证和信号配置，同时创建 COW 地址空间。共享地址空间创建期间若需要修改已有映射权限，内核会配合远端 TLB 刷新后再允许子任务运行。

CLONE_VFORK 建立一条一次性的父子同步关系：父任务在子任务 execve 成功装载新映像或退出之前保持阻塞；子任务通过 take_vfork_parent 只取得一次父任务标识，再唤醒父任务。这样既符合 vfork 的可见语义，也避免子任务多次退出路径重复唤醒父任务。execve 还会终止同一线程组中其余执行流，等待其停止后再提交新地址空间，防止旧线程继续访问已被替换的用户映像。

5.4 进程层级、组与资源边界

父子关系是 waitpid、孤儿进程收养和 SIGCHLD 递送的基础。ScintillaOS 通过 TCB 中的父引用和子表维护进程层级；父进程退出后，子任务交由 init 进程收养，使后续退出状态仍然有明确回收者。工程中定义了 setpgid、getpgid 和 setsid 的入口，但进程组、会话和作业控制并不是当前兼容能力的重点，相关行为需要以实际测试覆盖为边界。

线程组将“共享进程语义的多个任务”与一般父子关系区分开来。getpid 返回线程组号，单个执行流仍有独立的任务 PID；tgkill 按线程组和目标任务双重校验信号目标。等待路径默认允许同一线程组内的任务共同观察适用的子进程退出状态，而 __WNOTHREAD 可收窄为仅等待调用任务的直接子进程。退出通知也会唤醒同一线程组内可能执行等待操作的成员，避免把等待责任错误绑定到某一个

内核线程。

资源边界则用于防止异常程序无限占用内核对象。fd 表设置最大描述符数量，用户栈和内核堆也采用固定配置。对共享 fd 表而言，资源边界不仅是数量限制，还要求操作的可见性一致：文件表组锁覆盖表项替换、引用计数调整及 close-on-exec 扫描；底层 `File` 对象仍以 `Arc` 引用管理。地址空间共享、fd 表共享和 CPU 亲和性分别由不同字段明确表达，内核不会因两个任务同属线程组就隐式共享全部资源。

5.5 进程生命周期

进程生命周期可以看成一条从创建、执行、替换到退出回收的状态链。`fork` 复制进程状态并使用写时复制地址空间；`clone(CLONE_VM)` 可共享页表；`exec` 重新装载 ELF，同时保留未设置 close-on-exec 的文件描述符；`exit` 使进程停止运行并进入 Zombie 状态。这样设计的原因在于，父进程需要在子进程退出后读取退出码，内核不能在子进程停止执行的瞬间立即释放所有元数据。

当前任务状态覆盖 `Ready`、`Running`、`Switching`、`Blocking`、`Blocked`、`WakePending` 和 `Zombie`。其中前六种状态服务于每 CPU 调度和并发唤醒，`Zombie` 表示执行流已经停止但退出信息仍对等待者可见。`exit_group` 会对同一线程组内其他非僵尸任务发出终止请求并唤醒其可能所在的 CPU；普通退出仅结束当前任务。

退出发布采用两阶段策略。`begin_exit` 先一次性完成退出事件和退出码的权威发布，并设置 `waitable_zombie`，让等待者可以在资源清理尚未结束时可靠观察到可领取的退出信息；随后再进行可能较慢的文件表拆除、地址空间句柄释放和最终 CPU 交接。该顺序缩短了持锁区间，也保证父任务被唤醒后不会读取到半更新的退出状态。

5.6 信号机制

信号机制使进程能够接收异步事件，例如父子退出通知、用户终止请求或非法访问导致的致命事件。ScintillaOS 将待处理信号以位图形式保存在进程控制块中，并在陷阱返回用户态前统一处理。内核实现 `kill`、`tgkill`、`sigaction`、`sigprocmask` 和 `sigreturn` 等接口；默认动作包括终止、停止、继续或忽略。安装用

户处理函数时，内核备份 TrapFrame，将 PC 改为处理函数入口，并在 sigreturn 中恢复原上下文。

信号的待处理集合、阻塞掩码和动作表在 TCB 资源锁内分别维护；递送路径在选出一个未屏蔽信号后构造用户态信号帧，并通过受控的用户指针翻译写入栈。用户内存访问不在持有任务资源锁时执行，避免缺页或页表锁与信号锁交叉。SIGKILL 和 SIGSTOP 不受普通屏蔽掩码影响；致命信号、停止和继续信号会同步改变任务的调度可运行性。

面向线程组的 kill 与面向具体任务的 tgkill 共享同一套目标存活性校验。信号到达阻塞任务时，内核通过统一唤醒状态机使其回到合适 CPU 的就绪队列；若信号和睡眠条件并发发生，最终状态由再次检查和原子状态转换决定。因而信号不仅是 TrapFrame 修改逻辑，也是进程状态机和 SMP 唤醒机制的一部分。

5.7 退出、回收与等待

退出路径是进程管理中最容易跨模块出错的部分。当前任务调用 exit 或因致命信号退出时，内核需要同时更新进程状态、关闭文件描述符、回收非共享地址空间、处理孤儿进程并通知父进程。若父进程正在等待子进程，内核唤醒等待队列；若 SIGCHLD 未被屏蔽，则在父进程待处理信号集中置位。waitpid 最终读取退出码，并完成僵尸进程对象的回收。

因此，“退出”和“回收”必须区分为两个阶段：子进程先停止执行并保留退出状态，父进程随后通过 wait 类调用取得状态并完成最终回收。这个划分避免父进程尚未读取退出信息时丢失子进程状态，也让信号递送和等待队列唤醒有明确的同步点。

代码5.2根据 os/src/task/mod.rs:384--451 整理了退出路径。它展示了退出操作不仅是设置返回码，还必须同步处理地址空间、文件描述符、亲缘关系和等待者。

代码 5.2　当前任务退出与僵尸化处理

```
1 pub fn exit_current_and_run_next(exit_code: i32) -> ! {
2     crate::fs::stdio::stdout_release_if_owner(); // 释放当前进程的终端输出
          所有权
3     let task = current_task().unwrap();
4     remove_from_ready_queue(task.getpid());
```

```
5     remove_from_pid2task(task.getpid());
6     let (parent, orphan_children) = {
7         let mut inner = task.inner_exclusive_access();
8         inner.task_status = TaskStatus::Zombie; // 保留退出状态，等待父进程
              回收
9         inner.exit_code = exit_code;
10        let parent = inner.parent.as_ref().and_then(|w| w.upgrade());
11        let orphan_children = core::mem::take(&mut inner.children);
12        inner.memory_set.recycle_data_pages(); // 回收非共享用户页
13        inner.clear_fd_table(); // 关闭所有文件描述符
14        (parent, orphan_children)
15    };
16    if let Some(parent) = parent {
17        notify_parent_sigchld(&parent); // 通知父进程存在已退出子进程
18        let mut queue = WAITPID_QUEUE.exclusive_access();
19        if let Some(idx) = queue.iter().position(|t| t.getpid() == parent.
              getpid()) {
20            let parent_task = queue.remove(idx);
21            drop(queue);
22            parent_task.inner_exclusive_access().task_status = TaskStatus::
                  Ready;
23            add_task(parent_task);
24        }
25    }
26    if !orphan_children.is_empty() {
27        let mut init = INITPROC.inner_exclusive_access();
28        for child in orphan_children.iter() {
29            // 孤儿进程由 init 进程收养
30            child.inner_exclusive_access().parent = Some(Arc::downgrade(&
                  INITPROC));
31            init.children.push(child.clone());
32        }
33    }
34    if let Some(current) = take_current_task() {
35        defer_drop_task(current);
36    }
37    loop { schedule_idle_context(); }
38 }
```

实现首先取得当前 TCB，并把任务状态置为 Zombie；随后回收不与线程共享的用户页，关闭 fd 表。父子关系更新与 SIGCHLD 递送保证父进程能够通过 waitpid

观察退出状态。对于等待子进程或 futex 的任务，唤醒逻辑必须在状态更新之后执行，否则父进程可能被唤醒却看不到一致的僵尸状态。由此可见，退出路径不是单独的进程逻辑，而是进程、内存、文件和信号机制共同维护的一致性边界。

6 文件系统模块

6.1 虚拟文件系统架构

文件系统模块要解决的核心问题，是让用户程序通过统一路径和 fd 接口访问不同来源的对象。早期直接围绕单一 Ext4 实例和字符串路径实现文件访问，无法表达挂载点，也难以支持多个文件系统。例如测试盘挂载到 `/mnt` 后，`/mnt/a` 应访问另一块设备，而不是根盘目录。因此 ScintillaOS 引入 VFS 子层，用 `VfsSuperBlock` 表示已挂载文件系统实例，用 `Dentry` 表示路径分量，用 `Mount` 记录挂载点和被挂载根节点的对应关系。

VFS 的意义在于把“路径解析”和“具体 Ext4 调用”分离开来。路径解析逐级调用 `vfs_lookup`，命中 Dentry 缓存时直接返回，未命中时读取 Ext4 目录项并填充缓存；当路径进入挂载点时，解析过程透明切换到被挂载文件系统的根 Dentry。这样，`cwd`、`root` 和文件描述符引用的是稳定内核对象，而不是容易失效的字符串路径。

当前 VFS 面向 SMP 重新划分了锁域。每个 `VfsSuperBlock` 有自己的文件系统锁，但 lwext4 内部仍包含进程级的挂载和缓存状态，所以所有直接 lwext4 操作还必须经过全局 `LwExt4Global` 锁。路径缓存、目录项变更、inode 运行时文件大小和挂载表各自使用独立锁，文件系统 I/O 不会再依赖单核 `UPSafeCell` 的隐式互斥。

6.2 核心对象与对象关系

对象	设计与职责
`VfsSuperBlock`	一个已挂载文件系统的实例，内部保存文件系统类型和 Ext4 私有对象；由 `Arc` 共享。
`Dentry`	名字到文件系统对象的缓存节点，保存 `name`、统一命名空间路径、文件系统内部路径、父节点、子节点缓存和所属超级块。

对象	设计与职责
`Mount`	将父文件系统中的挂载点 Dentry 映射到新文件系统根 Dentry。
`OSInode`	一个打开文件句柄，保存可读/可写标记、Dentry 和当前偏移；承担 ScintillaOS 中 file/inode 的主要运行时职责。
`File`	统一 I/O trait。普通文件、Pipe、Stdin、Stdout 和部分特殊文件以同一方式进入 fd 表。
Dentry 一致性元数据	子节点缓存由 `BTreeMap` 保存，并配有目录变更锁、缓存代际、父目录验证代际和负项缓存。目录变更期间代际为不稳定值，慢速 Ext4 查找在锁外进行，返回后才短暂提交缓存。
`InodeRuntimeSize`	以“超级块地址、inode 号”为键维护运行时文件大小、缓存 Ext4 文件句柄和待写数据，供多个打开实例观察一致的逻辑文件长度。
打开文件描述	`OSInode` 的偏移、目录游标、追加写标志和底层句柄由睡眠锁保护；它代表一次打开，而不是仅代表磁盘 inode。

表中的对象共同构成 VFS 的分层边界：超级块负责文件系统实例，Dentry 负责命名空间节点，Mount 负责跨文件系统跳转，`File` trait 负责打开后的统一 I/O 行为。代码6.1是 `os/src/fs/vfs.rs:545` 中 Dentry 的结构摘录。它同时保存面向用户的统一路径 `path` 与交给 Ext4 的内部路径 `fs_path`；二者在根文件系统中通常一致，但在 `/mnt` 等挂载点之后会不同。

目录项缓存不再只缓存正向查找结果。不存在的名称会进入负项缓存，直到所属目录发生创建、删除、重命名等变更才失效；已有子节点则以目录代际验证其仍属于当前父目录。这样可避免大型构建目录中对同一缺失文件反复进入 Ext4，同时不会让缓存结果跨目录变更长期失真。

代码 6.1　Dentry 的关键字段

```
pub struct Dentry {
    pub name: String,
    pub path: String, // 统一命名空间中的路径
```

```
4     pub fs_path: String, // 所属文件系统内部路径
5     pub inode_num: u32,
6     pub is_dir: bool,
7     pub is_link: bool,
8     pub parent: Option<Weak<Dentry>>, // 使用弱引用避免目录树循环引用
9     pub super_block: Arc<VfsSuperBlock>, // 决定底层 Ext4 实例
10    pub children: UPSafeCell<Option<Vec<Arc<Dentry>>>>, // 惰性子目录缓存
11 }
```

第 2–4 行将“名字”“全局路径”和“文件系统内部路径”分离，避免把挂载逻辑退化为字符串拼接。第 7 行使用弱引用指向父节点，避免 Dentry 树形成引用环；第 8 行决定每次 I/O 应下发到哪个 Ext4 实例；第 9 行的 None 表示子目录尚未从磁盘装入，Some 则保存已缓存的子节点。该缓存主要服务于路径查找，并不等同于完整页缓存。

6.3 路径解析与文件打开

路径解析之所以需要单独说明，是因为它决定了用户命名空间如何映射到底层磁盘对象。绝对路径从任务的 root 开始，相对路径从 cwd 开始；解析器逐项处理“.”、“..”和普通路径分量。其中“..”必须感知挂载根，避免从被挂载文件系统根节点直接跳到错误父节点；普通分量则先查询 Dentry.children 缓存，未命中才调用 Ext4 获取目录项并建立缓存。

代码6.2根据 os/src/fs/vfs.rs:688--735 整理。它说明路径解析并非简单拼接字符串：每一个路径分量都可能命中缓存、触发底层目录读取，或因为自身是挂载点而跳转到另一个超级块。

代码 6.2　单级 Dentry 查找和挂载点穿越

```
1 pub fn vfs_lookup(parent: &Arc<Dentry>, name: &str) -> Option<Arc<Dentry
      >> {
2     match name {
3         "." => return Some(parent.clone()), // 当前目录
4         ".." => return vfs_parent(parent).or_else(|| Some(parent.clone()))
              , // 父目录
5         _ => {}
6     }
7     {
8         let children = parent.children.exclusive_access(); // 优先查询
```

```
            Dentry 缓存
        if let Some(list) = children.as_ref() {
            if let Some(cached) = list.iter().find(|c| c.name == name) {
                if let Some(mnt_root) = lookup_mount(cached) {
                    return Some(mnt_root);
                }
                return Some(cached.clone());
            }
        }
    }
    let child_fs_path = if parent.fs_path == "/" {
        format!("/{}", name)
    } else if parent.fs_path.ends_with('/') {
        format!("{}{}", parent.fs_path, name)
    } else {
        format!("{}/{}", parent.fs_path, name)
    };
    let info = parent.super_block.get_inode_info(&child_fs_path)?; // 缓存
        未命中，访问 Ext4
    let child = Dentry::new_child(parent, name, info.inode_num,
                                  info.is_dir, info.is_link);
    cache_child(parent, child.clone()); // 将新节点放入缓存
    if let Some(mnt_root) = lookup_mount(&child) {
        return Some(mnt_root); // 挂载点透明跳转到新文件系统根目录
    }
    Some(child)
}
```

对于 “.”，查找直接返回当前节点；对于 “..”，实现转交给感知挂载关系的父目录逻辑。普通名称先查询 children 缓存，若缓存未初始化，再通过 super_block 调用 Ext4 读取目录项。查到节点后还需检查它是否为 MOUNT_TABLE 中的挂载点：如果是，返回挂载文件系统根目录。因此，用户对 /mnt/foo 的 openat 不需要知道 /mnt 前后对应不同磁盘。

openat 最终调用 open_file_new。带 O_CREATE 标志时，内核先检查目标是否存在，再决定打开、截断或创建；常规打开则解析路径并创建 OSInode。/proc 和 /dev 前缀在进入 Ext4 查询之前由专用逻辑截获，因此它们不依赖磁盘中的同名文件。这样，普通文件、特殊文件和挂载文件系统都能进入同一 fd 表。

6.4 Ext4 与双盘设计

Ext4 负责提供真实磁盘文件语义，VFS 负责把这些语义纳入统一命名空间。ScintillaOS 的底层 Ext4 功能由 `lwext4_rust` 提供，VFS 通过 `with_fs` 闭包在受保护的 Ext4 实例上执行读写、目录枚举、创建、截断和同步操作。块设备层将 VirtIO 块设备包装为连续数据流，再交给 lwext4 使用。这样的分层使文件系统逻辑不需要直接处理 VirtIO 队列细节。

双盘设计来自比赛评测需求。bus0 对应评测提供的测试盘，bus1 对应根文件系统镜像；启动时 `mount_root` 使用根盘，`sys_mount` 可将测试盘挂载到目标目录。这样可以避免测试数据与系统根文件混用，也使 `/mnt` 下的访问路径与评测场景保持一致。图6.1展示了用户命名空间、超级块和 VirtIO 块设备之间的关系。

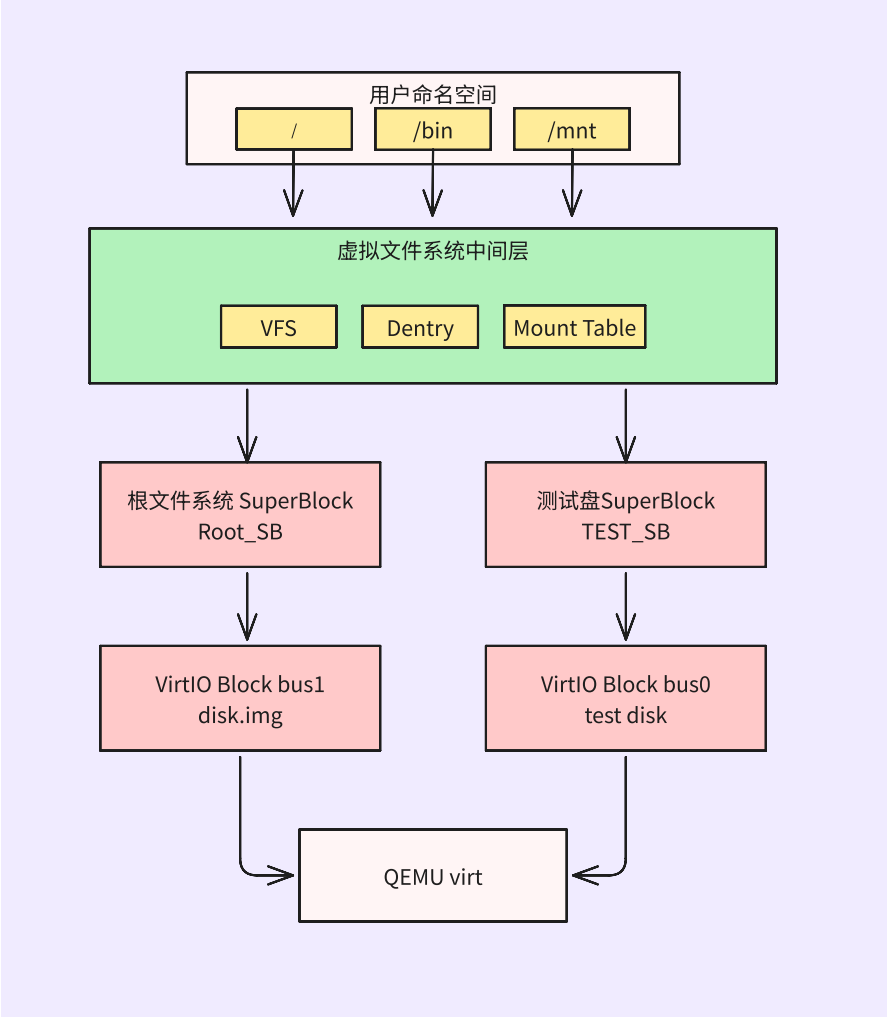


图 6.1　VFS 与双盘结构

双盘并发访问是文件系统 SMP 化的关键约束。尽管每个卷拥有独立 `VfsSuperBlock`，lwext4 的全局状态意味着根盘读写、测试盘挂载和另一个卷的目录操作不能并发进入其 C 库；内核因此按“卷锁后进入全局 lwext4 锁”的顺序串行化直接调用。块设备访问和可能阻塞的 I/O 不携带 Dentry 缓存或任务资源锁，避免文件系统操作与调度、缺页和进程退出形成循环等待。

普通文件写入采用运行时文件大小和待写范围缓存：连续小写可合并为区间，达到单 inode 或全局压力阈值时执行回写；关闭、截断、同步及最终 unlink 也会触发相应冲刷或失效。文件内容被修改后，内核会使内存管理模块中的文件页缓存失效或更新，避免 `mmap` 和普通 `read` 观察到不同版本的数据。

6.5 挂载、卸载与一致性

挂载操作改变的是命名空间，而不是简单改变字符串前缀。执行 mount 系统调用时，内核先将目标路径解析为挂载点 Dentry，再为块设备创建新的 `VfsSuperBlock`，最后向全局挂载表加入挂载点与挂载根目录的对应关系。后续路径解析遇到该挂载点时，会转入新超级块根目录。卸载则需要删除相应挂载关系，并避免留下不一致的命名空间映射。

挂载表预留固定容量，插入时采用“锁内预检、锁外构造超级块、锁内复检并发布”的协议。这样既不在 VFS 锁内执行可能耗时的 Ext4 挂载，也避免两个 CPU 同时向同一挂载点发布不同根目录。当前 `mount` 支持 Ext4，并为评测兼容接受特定 VFAT 场景；`tmpfs` 挂载记录路径上的容量限制和只读策略，`MS_REMOUNT` 可以更新只读状态。

卸载会从挂载表移出目标并强制卸载其 Ext4 实例，同时清除该路径的 tmpfs 兼容状态和只读策略。关机前，内核会收集去重后的已挂载超级块，先处理子挂载卷，再冲刷和卸载根卷。Dentry、已打开 `OSInode` 和进程 cwd/root 都以 `Arc` 保持对象生命周期，因此路径缓存的释放与底层卷的卸载分开处理；当前实现以评测环境的受控挂载生命周期为边界，不将其宣称为完整的 Linux busy-unmount 语义。

6.6 特殊文件和 IPC

特殊文件和 IPC 说明了 VFS 不只是 Ext4 的封装。ScintillaOS 提供 `/proc/mounts`、`/proc/meminfo`、`/proc/<pid>/stat` 等 proc 风格接口，以及 `/dev/null`、`/dev/zero`、`/dev/urandom` 和 `/dev/rtc` 等设备文件。管道采用 4 KiB 环形缓冲区，支持阻塞和非阻塞读写。它们的底层实现各不相同，但都通过 `File` trait 进入 fd 表，因此系统调用层可以用同一套 read/write/close 语义处理普通文件、标准输入输出、管道和设备对象。

伪文件接口扩展了 `/proc/cpuinfo`、`/proc/uptime`、`/proc/self/status`、`/proc/self/maps`、`/proc/sys/kernel` 及部分 `/sys` 块设备信息；这些内容在打开或读取时根据当前任务、CPU 和挂载表生成，不占用 Ext4 inode。`/dev/random` 与 `/dev/urandom`、`/dev/misc/rtc` 也按字符设备对象返回，满足用户态运行时和工具对路径、`stat` 与可读性的基本预期。

管道使用 16 页环形缓冲区而非固定 4 KiB 缓冲区，读端和写端共享一个设备锁保护的缓冲对象，并记录两端打开计数、非阻塞标志和 poll 就绪状态。读空管道、写满管道、端点关闭、`F_SETFL(O_NONBLOCK)`、信号中断与跨 CPU 唤醒均通过第三章的统一阻塞状态机处理。`eventfd`、`epoll` 和 Unix 域套接字也实现 `File` trait，使它们能够与普通 fd 一起参与 `poll`、`select` 和 close-on-exec 语义。

7 信号处理模块

7.1 信号发送与待处理状态

信号机制解决的是进程之间异步事件传递的问题。子进程退出、用户请求终止程序、非法内存访问或外部唤醒，都不能简单地等到目标进程下一次主动系统调用再处理。ScintillaOS 通过进程控制块中的待处理位图记录信号，`kill` 和 `tgkill` 根据 PID/TID 找到目标任务后，将信号置入目标的待处理集合。标准信号在位图模型中会合并；对于 `SIGKILL`、`SIGTERM` 等致命信号，内核同时更新终止状态。

仅记录信号还不够，因为目标任务可能正阻塞在 futex、wait、网络 I/O 或睡眠队列中。如果发送信号后不唤醒目标，目标就无法回到内核检查待处理状态。因此，信号发送路径与调度和阻塞队列存在直接依赖：发送端负责让目标具备再次运行的机会，真正的信号递送则推迟到目标即将返回用户态之前完成。这样可以避免在任意内核路径中强行改写用户现场。

当前实现以 `SignalFlags` 位集保存 1 至 34 号可识别信号，并为每个信号槽保留 `PendingSignalInfo`，记录发送原因、发送者进程号和有效 uid。同一信号已处于待处理状态时，新的标准信号与其合并而非重复排队；因此该设计明确提供标准信号的合并语义，而不把实时信号误描述为完整的 FIFO 实时信号队列。

`kill` 支持正 PID、当前进程组、指定进程组和全体可见任务的目标形式，`tkill` 以 TID 定位单一任务，`tgkill` 则同时核验 TGID 与 TID，避免线程标识复用时向错误线程组递送。信号发送只在短暂持有目标任务锁时更新待处理位、元数据和致命状态；随后通过统一 `wake_task` 状态机处理阻塞目标。若目标处于 `Blocking`，唤醒会先发布 `WakePending`，由原 CPU 在上下文保存完成后再入就绪队列，从而避免跨 CPU 直接运行尚未保存完整上下文的任务。

`SIGKILL` 不能被安装处理函数或加入屏蔽掩码，并会触发目标任务及其线程组的退出收敛；一般可捕获信号只置入待处理集合，是否终止由已安装动作或默认动作决定。管道和 Unix 域套接字在向无读端写入时也会向当前任务加入 `SIGPIPE`，缺页、非法指令等异常路径则通过同一待处理集合提交 `SIGSEGV`、`SIGILL` 等同步信号。

7.2 信号递送时机

ScintillaOS 选择在陷阱返回用户态前递送信号。这个时机比较自然：内核已经拥有当前任务的 TrapFrame，可以安全修改用户态 PC、参数寄存器和用户栈；同时，任务尚未真正回到用户态，不会出现用户代码执行一半时内核突然插入处理函数的情况。递送逻辑会遍历待处理信号，选择未被信号阻塞掩码屏蔽的项；若信号具有用户处理函数，内核保存当前 TrapFrame，设置处理函数入口，传递信号编号并配置返回跳板；若使用默认动作，则按信号类型执行终止、停止、继续或忽略。

`sigreturn` 是用户处理函数返回内核的约定入口，用于恢复此前保存的执行上下文。因此，信号帧既要能让用户处理函数获得信号编号，又要保存旧掩码和被中断 PC。代码7.1摘自 `os/src/task/mod.rs:619` 附近的信号帧构造逻辑，展示了内核如何在用户栈中预留 `siginfo` 与 `ucontext` 空间。

代码 7.1　用户态信号帧的构造摘录

```
1  fn write_signal_frame(token: PageTable, sp: usize, sig: usize,
2                        old_mask: SignalFlags, interrupted_pc: usize) -> usize
                             {
3      let frame_size = SIGINFO_SIZE + UCONTEXT_SIZE;
4      let frame_base = sp.saturating_sub(frame_size) & !15;
5      for i in 0..frame_size { // 清零用户栈中的信号帧
6          *translated_refmut(token, (frame_base + i) as *mut u8) = 0;
7      }
8      *translated_refmut(token, frame_base as *mut i32) = sig as i32;
9      let ucontext = frame_base + SIGINFO_SIZE; // siginfo 之后保存 ucontext
10     *translated_refmut(token, (ucontext + UCONTEXT_SIGMASK_OFFSET) as *mut
           u64)
11         = flags_to_user_sigset(old_mask);
12     *translated_refmut(token, (ucontext + UCONTEXT_PC_OFFSET) as *mut
           usize)
13         = interrupted_pc;
14     frame_base
15 }
```

第 3 行向下扩展用户栈并按 16 字节对齐；第 4–6 行先清空帧内容，避免旧数据暴露给用户态；第 7–12 行填入信号编号、旧掩码和中断 PC。用户处理函数结束后，跳板执行 `sigreturn`，内核再根据这些信息恢复现场。该路径同时涉及用户

地址写入、TrapFrame 修改和异常返回，必须和内存模块的用户指针检查、进程模块的阻塞唤醒保持一致。

7.3 信号处理的限制

信号处理的复杂性在于它跨越多个边界：发送阶段涉及 PID 查找和阻塞队列唤醒，递送阶段涉及用户栈布局和 TrapFrame 修改，返回阶段又依赖 `sigreturn` 恢复现场。ScintillaOS 已实现 `sigprocmask`、`rt_sigtimedwait` 等接口和基础递送路径，但复杂多进程信号同步、系统调用被中断后的返回值处理以及部分 LTP 场景仍属于风险区域。因此，信号模块的结论应区分“基础 kill/sigaction 路径可用”和“完整 POSIX 信号语义仍需持续验证”。

8 系统调用与用户态支持

8.1 系统调用接口

系统调用层是用户态程序观察内核能力的 ABI 边界。ScintillaOS 的系统调用入口位于 `os/src/syscall/mod.rs`，按 RISC-V Linux ABI 从寄存器接收最多 6 个参数，并将执行结果写回 TrapFrame 的返回寄存器。选择这一分层的原因在于，系统调用入口本身不应承载全部业务逻辑；它只负责解码调用号、读取参数并把请求转交给真正拥有状态的子模块。

ScintillaOS 覆盖的接口类别包括进程、文件、内存、信号、IPC、凭证、时间和网络等。正文以接口类别和代表性调用说明设计关系，完整清单通过表格呈现关键接口。代码8.1摘自 `os/src/syscall/mod.rs:180` 的分发表，展示了系统调用层如何把寄存器参数转换为 Rust 函数所需的整数、用户指针或结构体指针。

代码 8.1　文件系统系统调用分发摘录

```
let ret = match syscall_id {
    SYSCALL_OPENAT => sys_openat(
        args[0] as isize, args[1] as *const u8,
        args[2] as u32, args[3] as u32),
    SYSCALL_CLOSE => sys_close(args[0]),
    SYSCALL_GETDENTS64 =>
        sys_getdents64(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_READ => sys_read(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_WRITE => sys_write(args[0], args[1] as *mut u8, args[2]),
    SYSCALL_PIPE2 => sys_pipe2(args[0] as *mut usize, args[1]),
    _ => -38, // 未实现的系统调用返回 ENOSYS
};
```

例如，第 2–4 行将 `openat` 的目录 fd、用户态路径指针、打开标志和创建权限传给文件子系统；用户指针不会在分发层被直接解引用，而是在具体实现中通过页表转换和用户缓冲区检查访问。第 6–8 行显示读目录和普通读写虽然都经由 fd，但语义分别由 `getdents64` 和 `File` trait 处理。末行的 `-38` 对应 `ENOSYS`，用于让未实现接口以可诊断方式失败，而不是伪造成成功。

表 8.1　代表性系统调用类别

类别	代表性接口
进程	fork、clone、exec、waitpid、exit、brk、nanosleep
文件系统	openat、read、write、lseek、getdents64、mount、umount2、fsync
内存	mmap、munmap、mprotect、msync、mremap
信号	kill、sigaction、sigprocmask、sigreturn、rt_sigtimedwait
网络	socket、bind、listen、accept、connect、sendmsg、recvmsg

8.2　系统调用支持

Scintilla 实现的系统调用数量较多。我们按按接口归类，将实现的系统调用撰写在表8.2，并突出每类调用依赖的内核模块和当前支持状态。其中，“已实现”表示存在主要语义路径，“最小语义”表示接口为兼容特定程序提供受限行为，“占位或风险”则说明该调用不能作为完整兼容能力声明。

类别	代表接口（调用号）	支持级别	主要实现位置	说明
进程	fork(401)、clone(220)、execve(221)、waitpid(260)、exit(93)	已实现	`syscall/process.rs`、`task/task.rs`	fork 使用 COW；clone 根据标志共享资源。
文件 I/O	openat(56)、close(57)、read(63)、write(64)、lseek(62)	已实现	`syscall/fs.rs`、`fs/inode.rs`	通过 VFS/`File` trait 分发。
目录与元数据	getdents64(61)、fstat(80)、mkdirat(34)、renameat2(276)	已实现	`syscall/fs.rs`、`fs/vfs.rs`	依赖 Ext4 路径级操作。
描述符与 IPC	dup(23)、dup3(24)、fcntl(25)、pipe2(59)	已实现/部分	`syscall/fs.rs`、`fs/pipe.rs`	fcntl 仅覆盖当前测试所需命令。
内存	brk(214)、mmap(222)、munmap(215)、mprotect(226)、msync(227)	已实现/风险项	`syscall/process.rs`、`mm/memory_set.rs`	COW、保护页和复杂 LTP 场景仍需回归。
信号	kill(129)、sigaction(134)、sigprocmask(135)、sigreturn(139)	已实现/部分	`syscall/process.rs`、`task/signal.rs`	复杂用户态处理器返回路径仍是风险点。
时间与系统	nanosleep(101)、clock_gettime(113)、uname(160)、getrusage(165)	已实现/最小语义	`syscall/process.rs`	部分接口仅返回评测需要的字段。
凭证	getuid(174)、setuid(146)、getgroups(158)、umask(166)	已实现/部分	`syscall/process.rs`、`fs/vfs.rs`	权限语义受 Ext4 实现和测试覆盖限制。
网络	socket(198)、connect(203)、sendmsg(211)、recvmsg(212)	已实现	`net/syscall.rs`、`net/socket.rs`	支持 TCP/UDP 基础路径和环回。
共享内存	shmget(194)、shmat(196)、shmdt(197)	需复测	`syscall/process.rs`	不以本表作为完整 System V IPC 兼容性声明。
事件接口	epoll_create1(20)、eventfd2(290)、rseq(293)	占位/未实现	`syscall/mod.rs`	返回 `-ENOSYS` 或仅保留入口。

这张表需要和评测日志一起理解。增加一个调用号常量或 match 分支，并不等于该调用在错误码、阻塞语义和边界条件上都与 Linux 一致；真正的兼容性取决于它能否在对应模块中完成完整语义。例如 `mmap` 依赖虚拟内存和文件系统，`waitpid` 依赖进程生命周期，socket 调用则依赖网络端点和 fd 生命周期。

8.3 用户态运行时

用户态运行时为测试程序提供进入内核的最小支持。ScintillaOS 的 `user_lib` 提供 `_start` 入口、8 MiB 用户堆和系统调用包装，原始系统调用通过 RISC-V `ecall` 指令发起。选择静态链接策略，是为了降低早期 bring-up 阶段对动态加载器和共享库环境的依赖，使 shell、基础测试启动器、LTP 启动器和网络测试启动器能够先围绕内核 ABI 验证功能。用户态运行时虽然不属于内核核心，但它决定了测试程序如何构造参数、触发系统调用并解释返回值，因此也是 Linux 兼容链路的一部分。

9　网络模块

9.1　设计目标与分层架构

网络模块为用户态提供 Linux 风格的 socket fd。应用程序通过 `socket`、`bind`、`connect`、`listen`、`accept`、`sendmsg` 和 `recvmsg` 等系统调用访问端点；内核则把端点封装为实现 `File` trait 的对象，使其与普通文件、管道和设备文件共享 fd 表、`close`、`dup`、`fcntl`、`poll` 和 close-on-exec 的生命周期语义。

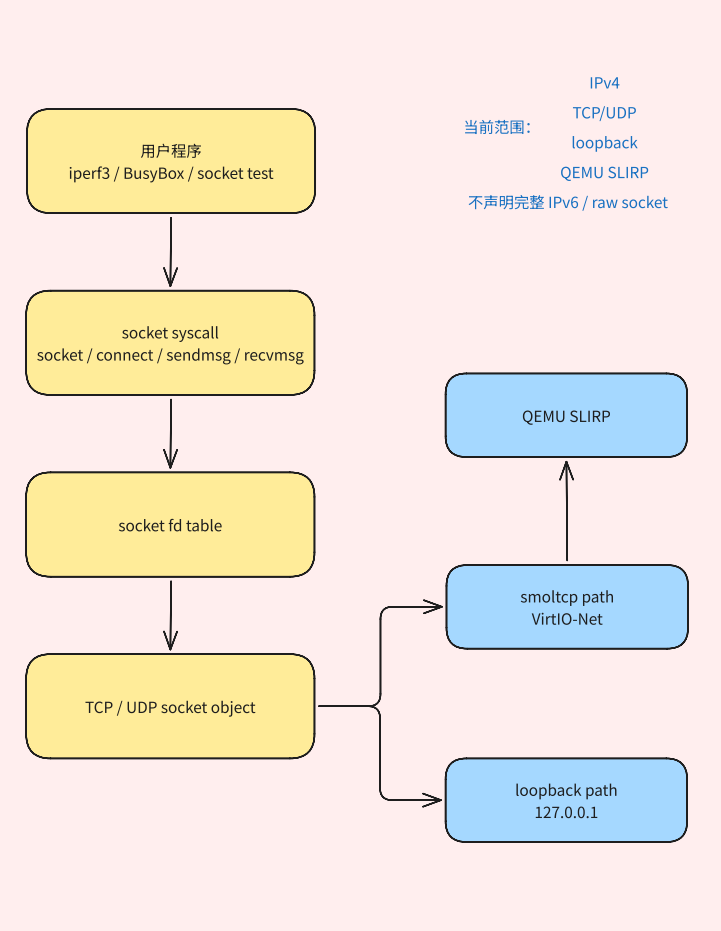


图 9.1　网络数据路径

除外部设备路径外，内核维护本地 TCP 监听表、双向连接缓冲区和 UDP 报文队列。对于环回或本地可达端点，连接与报文可以在内核内部完成转发，不依赖 VirtIO 设备是否存在；该路径和外部网络路径使用相同的 socket fd、阻塞、关闭和就绪通知语义，便于将 socket ABI 验证与 QEMU 网络环境解耦。

9.2 设备、协议栈与轮询模型

网络子系统位于 os/src/net。外部网络路径由 VirtIO-Net 适配器和 smoltcp 协议栈构成，NetworkState 将设备、smoltcp Interface、SocketSet、DHCP socket 和网络时钟组合为一个由 DeviceLock 保护的事务域。初始化时内核注册 127.0.0.1 环回地址，并提供适用于 QEMU SLIRP 环境的 IPv4 静态回退地址；DHCP 成功后可向接口补充获得的地址。

net_poll 是驱动协议栈进展的统一入口。时间维护路径会周期性调用它，connect、accept、ppoll 和 pselect6 等系统调用也会在检查就绪状态前主动轮询。轮询过程在 NET_IFACE 锁内推进 smoltcp 的时钟、设备收发和 socket 状态；离开该锁后才评估等待者并执行调度器唤醒操作，避免将设备锁延伸至任务状态和就绪队列。

该设计将“协议状态改变”和“任务可以再次运行”分为两个阶段。前者只修改接口和 socket 集合，后者依据等待原因检查具体条件并决定是否唤醒。这种分离既避免网络轮询期间持有调度器锁，也允许本地 TCP/UDP 路径在没有外部设备时复用相同的就绪判断接口。

9.3 Socket fd 与端点状态

每个 SocketFile 保存协议类型、smoltcp 句柄、对端和本地端点、可选的本地 TCP 连接标识、引用计数、非阻塞标志和监听状态。对象锁只保护这些元数据；进入 smoltcp、阻塞当前任务或唤醒其他任务前必须释放对象锁。NET_IFACE 也不与任务锁或调度器锁交叉持有，因此网络轮询、fd 关闭、进程退出和跨 CPU 唤醒不会围绕同一把锁形成循环依赖。

AF_INET 支持 SOCK_STREAM TCP 和 SOCK_DGRAM UDP。创建时解析 SOCK_NONBLOCK 与 SOCK_CLOEXEC；前者成为 socket 对象的打开状态，后者写入 fd 的 close-on-exec 位。SocketFile 还实现 get_flags、set_flags、dup_ref、close_ref 和 poll，从而使 F_SETFL、复制描述符和关闭最后一个引用的行为可与其他 File 对象统一处理。

关闭最后一个 fd 引用时，SocketFile 会撤销本地监听项、关闭本地 TCP/UDP 端点，并从 smoltcp 的 SocketSet 移除外部协议 socket。这样，socket 的协议资源回收以 fd 引用计数为边界，而不是依赖某一次特定系统调用路径；fork、CLONE_FILES 和 exec 时的 fd 操作仍由文件描述符表组同步机制协调。

9.4 TCP 与 UDP 传输路径

TCP 的 bind、listen、accept、connect、read、write 和 shutdown 围绕同一端点状态工作。外部连接由 smoltcp 的 TCP socket 维护；本地连接则在监听端口表中创建一对关联端点，客户端与服务端分别操作双向缓冲区。accept 对本

地监听项和 smoltcp 监听项采用一致的 fd 创建和地址返回语义。

UDP 可以绑定本地端口、通过 `connect` 记录默认对端，或通过 `sendto`、`sendmsg` 显式指定目的端点。发送时先判断目标是否由本地 UDP 队列接收，否则交给 smoltcp；接收时优先检查本地报文队列，再检查协议栈 socket。成功接收 UDP 报文时，`recvfrom` 或 `recvmsg` 可将源端点写回用户缓冲区。

`sendmsg`/`recvmsg` 支持从 iovec 聚集数据或向 iovec 分散数据，常用的 `MSG_DONTWAIT` 与 `MSG_PEEK` 分别影响等待和接收队列消费行为。TCP 与 UDP 的普通 `read`/`write` 最终也落到对应的发送、接收实现，因此应用程序可以按文件 I/O 或 socket I/O 两种风格使用同一端点。

对象或接口	主要语义	与 fd/调度器的关系
TCP socket	建立连接、监听与接受连接，支持流式读写、关闭和对端查询。	空接收、满发送缓冲区、尚未完成的连接或等待接受连接时可阻塞。
UDP socket	支持绑定、默认对端、定点收发和源端点返回。	无报文时阻塞或返回 `EAGAIN`；可通过 `poll` 观察可读性。
`sendmsg`/`recvmsg`	将 iovec 聚集到内核缓冲区后发送，或接收后分散写回 iovec；处理常用消息标志。	`MSG_DONTWAIT` 使操作不阻塞，`MSG_PEEK` 保留接收队列中的数据。
`SocketFile::poll`	汇总 smoltcp socket 或本地队列的可读、可写和错误状态。	为 `ppoll`、`pselect6` 和 `epoll_pwait` 提供统一的就绪性接口。
Unix `socketpair`	`AF_UNIX` 的 `SOCK_STREAM` 与 `SOCK_SEQPACKET` 端点对。	共享双向有界队列；关闭对端时写入触发 `SIGPIPE`，并支持非阻塞和 poll。

9.5 阻塞 I/O、轮询与跨核唤醒

网络 I/O 的阻塞路径与 SMP 调度器直接协作。`connect`、`accept`、TCP 发送/接收和 UDP 接收分别以 `TcpConnect`、`TcpAccept`、`TcpSend`、`TcpRecv`、`UdpRecv` 等原因登记等待。每个操作先检查协议状态或缓冲区条件，再发布阻塞状态并再次检查条件；非阻塞 fd 或 `MSG_DONTWAIT` 返回 `EAGAIN`，非阻塞 TCP 连接未完成时返回 `EINPROGRESS`。

net_poll 在定时路径和需要立即检查网络状态的系统调用中运行。它在 NET_IFACE 锁内推进 smoltcp，然后从调度器取得网络等待者快照，按等待原因检查连接、发送窗口、接收队列和本地监听队列是否已经满足条件。真正的重新入队由 wake_task 完成：若唤醒与任务切出竞争，任务先变为 WakePending，由原 CPU 完成上下文保存后再成为 Ready。因此，网络轮询不会跨 CPU 直接调度一个上下文尚未保存完整的任务。

SocketFile::poll 将外部 socket 与本地队列的可读、可写和错误状态统一为 File::poll 返回值，供 ppoll、pselect6 与 epoll_pwait 使用。事件等待在每次阻塞前重新取样状态；网络包到达、端点关闭、发送窗口释放或本地队列写入后，等待者通过同一唤醒状态机恢复执行。这使网络 fd 与管道、eventfd 和普通文件的事件模型保持一致。

9.6 本地 Unix 域通信与资源回收

本地 Unix 域通信目前以 socketpair 为主。两个端点共享一个设备锁保护的状态对象，分别拥有向对端发送的队列、字节计数和端点引用计数；SOCK_SEQPACKET 保留消息边界，SOCK_STREAM 则支持连续读取。该实现不依赖 smoltcp，但和管道、TCP socket 一样使用信号中断、阻塞重检和 fd 引用计数规则。

当对端全部关闭时，写端返回 EPIPE；若未指定 MSG_NOSIGNAL，内核同时向当前任务递送 SIGPIPE。队列满时写端阻塞或返回 EAGAIN，队列空时读端阻塞或返回相应状态；读写、关闭和 poll 的唤醒均按队列方向进行。由此，socketpair 既提供无外部设备依赖的本地 IPC，也复用了文件、信号和调度模块已经建立的并发约束。

9.7 支持范围与工程边界

ScintillaOS 的外部协议栈当前以 IPv4 为边界。VirtIO-Net 设备、QEMU 网络后端、DHCP 结果和静态地址配置都会影响端到端连通性，因此本地环回验证和外部网络验证必须分别对待：前者主要检验 socket 语义与阻塞唤醒，后者还依赖虚拟设备和网络环境。

当前不支持 IPv6、原始套接字、完整 Netlink、网络命名空间和防火墙/路由子系统；Unix 域套接字也尚未实现路径名 bind/connect 的命名空间，稳定路径是 socketpair。listen 的 backlog、部分 setsockopt/getsockopt 选项、控制消息以及高级拥塞控制只提供常见用户态程序所需的有限语义，不应视为完整 POSIX 或 Linux socket 选项集。

因此，本模块的能力定位为：通过统一 fd API 使用 IPv4 TCP/UDP、支持环回/本地传输、基本事件等待和常用 socket 操作。外部吞吐、长连接压力、丢包

重传以及多核高并发行为仍需要在目标架构、QEMU 后端和评测镜像上持续回归；网络设备不可用时，内核保留本地传输能力，并对不能建立的外部连接返回明确错误。

10 决赛负载适配

10.1 负载特征与适配目标

决赛负载不再只是独立系统调用的功能验证，而是以 glibc 用户态、Rust/-Cargo 工具链和多进程构建树为代表的长时间复合工作负载。一次真实构建会并发创建线程与子进程，频繁执行 `clone`、`execve`、`wait`、futex、信号和 fd 复制；同时持续进行路径解析、临时文件创建、重命名、截断、ELF 读取、内存映射和写回。因而“单个接口能返回正确结果”不足以说明系统可以承受决赛负载，内核还必须保证并发所有权、资源回收和跨模块锁顺序在长时间运行中保持一致。

本章的适配目标是让内核在不识别特定程序名称、不缩减 Cargo 工作量、不修改用户态时间语义的前提下，改善复杂负载的正确性、容量边界和可观测性。实现重点包括：多核下任务唯一运行、线程组与共享资源语义、按需 ELF 装载和文件页缓存、高频 VFS 元数据操作、可控的延迟写回以及针对事件等待的统一阻塞唤醒。相应优化均以通用内核对象为作用范围，而非针对某一个评测文件或命令设置特例。

10.2 双盘评测执行链

决赛运行环境采用根文件系统与官方测试盘分离的双盘模型。根盘提供内核启动所需的用户程序和基础环境，测试盘通过 `mount` 挂载后进入评测目录；VFS 以超级块、挂载点和 Dentry 将两块盘合入同一命名空间。这样，评测数据、glibc 工具链和构建产物可以与根文件系统隔离，重启或更换测试盘时也不会改变内核镜像本身。

研发分支 `main` 默认保留 Shell 入口，便于人工执行定向测试与故障定位；自动评测分支的 `initproc` 则挂载测试盘、切换至对应用户环境，顺序运行 CAgent 和官方 BuildStorm 脚本，最后关机。入口覆盖 Q2.1 与 Q2.2 的测试流程，但“脚本被执行”不等同于“完整负载通过”：每次结果都必须结合架构、CPU 数、内存大小、根盘/测试盘版本、QEMU 参数、串口日志和真实退出码判定。

10.3 多进程、多线程与 SMP 资源调度

Cargo 和编译器会同时产生大量可运行任务与阻塞等待者。内核使用每 CPU FIFO 就绪队列、在线 CPU 掩码、任务亲和性和 IPI 唤醒分散调度压力；`enqueued` 与 `running_cpu` 原子状态确保一个任务不会既在某个就绪队列中又被另一 CPU 执行。对正在运行的任务，亲和性变更请求重调度而不是直接搬运其上下文；对已阻塞任务，唤醒方按 `Blocking`、`WakePending`、`Blocked` 和 `Ready` 的状态协议

发布可运行性，避免编译负载中常见的 wait、futex、pipe 和 poll 丢失唤醒。

线程式负载依赖 `clone` 的资源组合而不是简单复制 TCB。`CLONE_VM` 共享地址空间，`CLONE_FILES` 共享文件描述符表组，`CLONE_SIGHAND` 共享信号动作，TLS 和线程组号按创建标志建立；普通 `fork` 则使用 COW 创建独立地址空间。`waitpid`/`waitid` 默认可以在同一线程组范围内观察和回收适用的子进程，支持“一个线程创建子进程、另一个线程等待”的运行时模式。`vfork`、exec 时终止同组其他线程，以及 `FD_CLOEXEC` 的处理共同避免构建工具在替换映像或关闭管道时留下失效的共享状态。

编译型负载同时放大了分配器竞争。内核保留启动堆，并在大内存配置下按可用内存启用更大的可增长堆区；动态区按平台 CPU 数分片，小对象优先由 CPU 本地缓存服务，释放时按地址归属回到对应分片。该策略减少任务、Dentry、VFS 缓存、页表元数据和 lwext4 相关对象在同一全局堆锁上的竞争，同时保持小内存 QEMU 配置下的保守容量划分。

10.4 按需装载、内存容量与页缓存

大型 Rust 工具链本身包含大量不会在启动路径立即执行的 ELF 段。普通 `execve` 只读取 ELF 头和程序头，为 `PT_LOAD` 建立文件后备的惰性 VMA；代码页、只读数据页和 BSS 区域在首次访问时分别按文件页或匿名页装入。用户主栈上限为 8 MiB，但只预分配放置 argv、envp、辅助向量和启动数据所需的顶部页面，后续向下访问按需扩展。这样避免启动多个 `cargo`、`rustc`、链接器和 build script 时重复装入未访问页面。

只读、可执行文件页以 `(superblock, inode)` 为键进入分片文件页缓存，并支持顺序小读的预读窗口；多个进程可以共享同一份工具链文本页。可写私有映射不进入该共享缓存，`fork` 继续使用 COW 隔离写入。文件写、截断、重命名、删除以及共享映射回写会使相应缓存范围失效；页表撤销后，内核在释放帧前完成活跃 CPU 的远端 TLB 刷新，避免其他核访问已经复用的页帧。

内核向用户态写回复杂系统调用结果时，先验证 VMA、准备跨页访问所需的匿名页或 COW 页，再执行复制。若并发 `fork` 在准备与写回之间写保护了共享页面，复制路径会重新准备并重试，而不会把这种内部竞态转换为随机 `EFAULT`。这对编译器大量使用 iovec、状态结构、管道缓冲区和子进程返回状态的场景尤为重要。

10.5 高频文件系统操作与写回控制

复杂构建对 VFS 的压力主要来自高频、并发且短生命周期的元数据操作。Dentry 以 `BTreeMap` 缓存正向子项，并记录负项、目录变更锁和缓存代际；查找未命中时在锁外访问 Ext4，返回后复核代际再提交缓存。创建、删除、重命名和链接操作

通过目录变更域串行化，使 O_CREAT|O_EXCL 的检查与创建不会被并发 namespace 修改穿插，也避免缓存将已删除或新建项长期误判。

工具链常以“创建临时文件、写入、重命名覆盖、保留已打开 fd”完成原子更新。对于最后一个目录链接已删除但仍被打开的普通文件，VFS 保留内部 orphan 名并维持打开引用，直至最后一个 OSInode 关闭后再执行最终 unlink；ftruncate、rename 和 close 则同步更新 inode 运行时大小、Dentry 缓存和文件页缓存。这些规则避免 fd 仍在使用时底层 inode 被提前回收，也使构建程序的临时文件协议保持可见性一致。

写入路径以每 inode 的待写范围缓存连续小写，并在单 inode 阈值、全局压力、fsync、sync、截断或关闭等时机冲刷。大内存配置增加内核元数据可用空间，但全局待写数据仍设有独立硬上限，防止扩大的内核堆被延迟写缓存无界占用。lwext4 的直接调用继续由每卷文件系统锁和全局 lwext4 锁串行化；这保留了底层库的非重入约束，同时把 Dentry 查找、用户复制和调度等待移出该临界区。

10.6 事件接口、时间与运行时兼容

现代语言运行时不会只使用阻塞 read/write。内核将 eventfd、epoll、管道、Unix socketpair、TCP/UDP socket 和 File::poll 纳入同一 fd 事件模型；ppoll、pselect6 和 epoll_pwait 在检查就绪性后才登记 PollWait，并在事件、信号或超时发生时按统一状态机唤醒。F_SETFL(O_NONBLOCK) 可以更新支持该语义的文件对象，避免运行时将临时不可读或不可写误判为永久故障。

线程库与构建脚本还依赖 futex、robust list、clear_child_tid、信号掩码、sigreturn、单调时钟和实时时钟。futex 按私有地址空间或共享物理页形成等待键，支持等待、唤醒、重排队和超时；信号帧使用用户栈与独立 trampoline 恢复被中断的上下文；时间模块将单调运行时间与实时时钟分离，供 sleep、interval timer、/proc/uptime、网络轮询和用户态 clock_gettime 共同使用。

10.7 验证策略与当前边界

决赛负载的验证采用分层方法：先以 smpstress、clonesmptest、affinitytest、futextest、mprotecttest 和文件系统定向测试检查单一不变量，再在双盘、多核、干净覆盖层环境中执行 BusyBox、libc、CAgent、工具链探测、最小 Cargo 工程和更长的 BuildStorm 链路。每次回归均应保存内核提交、输入镜像哈希、QEMU 启动参数、CPU/内存配置、串口日志与测试标记，以区分内核故障、测试盘变化和宿主资源抖动。

当前归档结果可支持“RISC-V 与 LoongArch 的八核 CAgent 功能用例、以及 BuildStorm Q2.1 工具链和最小 Rust 工程路径已完成”的表述。对于 Q2.2，RISC-V 已完成 tg-xtask 的 446 个预构建单元，复杂工程正式构建及产物启动仍

须在目标环境重新完成端到端验证；LoongArch 全量链路也尚未形成可结算结果。因此，本章说明的是为复杂负载所做的内核适配与验证方法，不将阶段性进度、单机耗时或局部日志外推为 Q2.2 已通过或获得性能分。

11 开发目标、计划与过程记录

ScintillaOS 的开发过程围绕“在 rCore 基础上扩展出可运行 Linux 用户程序的竞赛内核”展开。与普通课程实验相比，竞赛内核更强调跨模块语义一致性：一个系统调用能否通过测试，往往同时取决于进程生命周期、地址空间、文件描述符、路径解析、设备驱动和调度路径。初赛完成了单核 Linux ABI 基础能力；决赛阶段则将工作重心扩展到双架构 SMP、双盘自动评测和 Rust/Cargo 复杂构建负载。本章据当前研发分支和自动评测分支的记录，说明目标、阶段计划、实施过程、验证口径与协作证据。

11.1 开发目标与范围

ScintillaOS 的首要目标，是在 RISC-V 和 LoongArch 的 QEMU 平台上提供稳定的 Linux ABI 兼容路径，使静态链接及 glibc 用户程序能够通过标准系统调用访问进程、文件、内存、信号和网络能力。选择 RISC-V、LoongArch、Ext4、VirtIO 和 QEMU 作为主要目标，是因为它们与 rCore 基础、比赛评测环境和可复现调试流程保持一致，能够让团队把精力集中在内核语义扩展上。

围绕这一目标，系统将 rCore 的基础任务模型扩展为具有进程、线程、信号、文件描述符继承和地址空间语义的 Linux 风格对象；将单一文件系统扩展为由 VFS、Dentry、挂载点和 Ext4 超级块组成的双盘命名空间；并接入 VirtIO 块设备和网络设备。内存模块支持写时复制、`mmap`、按需 ELF 装载、文件页缓存与远端 TLB 刷新，事件接口覆盖管道、`eventfd`、`epoll`、Unix 域 `socketpair` 以及 TCP/UDP socket。

决赛阶段的目标不是将这些接口孤立地“实现出来”，而是在真实并发负载中维持其组合语义。当前内核采用每 CPU 本地状态和 FIFO 就绪队列、在线 CPU 掩码、任务亲和性、IPI 唤醒以及任务唯一运行所有权；当前配置支持 RISC-V 最多 8 个应用 CPU、LoongArch 最多 12 个应用 CPU。第一代 SMP 的优先目标是正确性、可运行性和可验证性，并不宣称已经具备成熟的全局负载均衡、NUMA 或实时调度能力；复杂 POSIX 语义和多核行为均以对应架构、核数和测试镜像下的实际结果为准。

11.2 阶段计划

表 11.1 开发阶段计划记录表

阶段	目标	主要工作	验证方式
阶段一	基础运行环境	启动、陷阱、进程切换、基础 syscall	basic 测试、串口日志
阶段二	文件系统迁移	Ext4、VFS、Dentry、双盘挂载	挂载与文件读写测试
阶段三	兼容性补齐	mmap、信号、fork/COW、pipe、proc/dev	BusyBox、Libctest、LTP 子集
阶段四	网络与性能	socket、环回、VirtIO-Net、iperf3、IOzone、Lmbench、Libcbench、lua	网络脚本、性能测试
阶段五	稳定化与文档	回归测试、日志整理、报告与答辩材料	最终提交日志与截图
阶段六	多核与决赛测例	SMP 调度、IPI、远端 TLB 刷新、双盘自动入口、CAgent 与 BuildStorm 负载适配	多核定向测试、官方脚本、串口日志与产物检查

11.3 完整开发过程描述

ScintillaOS 的开发由 rCore 原生，一步步适配测试用例的方式开发。开发初期的重点是保证基础执行稳定，并且能够执行二进制文件。因此，我们将 basic 与 busybox 测试用例所谓我们的测试目标。首先丰富了基础系统调用，并添加了用户栈，初始化页帧分配器和页表、建立陷阱入口、挂载根文件系统并装载 `/bin/initproc` 的过程。而针对评测，我们将 rCore 的原生文件系统替换成 ext4 文件系统并引入 VFS、Dentry、超级块和挂载表，把路径解析与具体文件系统实例分离，使 `/mnt` 后的路径能够切换到测试盘，同时仍然通过统一 fd 接口暴露给用户程序。

此时 ScintillaOS 已经可以通过 basic 与 busybox 的四种测试用例。随后我们转向 lmbench，lua，libctest 等对内核的运行稳定性，运行速度，语义完整度等有要求的多个测试用例。其中 fork 等进程有关操作因为反复报错占据了我们这部分开发的大部分时间。`fork` 需要复制用户现场和文件描述符表，但直接复制全部页面代价过高；ScintillaOS 因此学习 Linux 的思路，引入 COW，将父子进程初始页面共享为只读，并在写缺页时复制。信号、wait 和退出回收则进一步要求进程对

象在退出后保留必要状态，直到父进程读取退出码。随后，我们尝试让操作系统适配网络模块，编写了有关系统调用，并在 ltp 等测试用例中进行深度评测。

在此之后，系统调用、测试适配进入稳定化阶段。此阶段我们不再只是增加调用号，而是校准 ABI 参数、错误码、阻塞唤醒和边界条件，使用户程序能够通过更多测试用例。

进入决赛阶段之后，开发进入阶段六。单个全局 FIFO 队列被演进为每 CPU FIFO 就绪队列，调度器使用 `enqueued` 与 `running_cpu` 等状态约束任务不会同时被两个 CPU 运行。对正在切出的任务，唤醒方通过 `Blocking`、`WakePending`、`Blocked` 和 `Ready` 的状态协议发布可运行性；`wait`、futex、pipe、poll 和网络 I/O 复用这一协议，以关闭“检查条件后登记等待”之间的丢失唤醒窗口。地址空间记录活跃 CPU，在撤销映射或改变 COW 页权限时先完成远端 TLB 刷新，再回收页帧。

复杂 Rust 构建进一步放大了线程、文件系统和内存路径中的竞争。为此，系统补齐 `clone` 的共享地址空间、共享 fd 表和共享信号动作语义，支持线程组范围的 `waitpid/waitid`、`FD_CLOEXEC`、`vfork`、robust list 和 `clear_child_tid`；增加 `eventfd`、`epoll` 与 Unix 域 `socketpair`。文件系统针对“已打开但最后一个链接已删除”的普通文件维护 orphan 生命周期，修正 `ftruncate`、rename 和目录迭代的并发边界；内存系统加入文件页缓存、按需装载、动态内核堆分层和延迟写回上限。上述工作作用于通用内核对象，不按 crate 名称、评测路径或输出结果设置特例。

11.4 团队分工协作与证据

我们在 `Documents/Worklog.md` 和 git 记录了详细记录了本团队的开发流程和开发分工，具体总结如下：

1. **雷雨泽（队长）：基础文件系统、Ext4/lwext4 迁移与项目统筹**。雷雨泽作为队长，主要负责早期文件系统迁移、基础文件系统调用适配和整体开发推进。其工作包括将文件系统从 rCore 基线迁移到 Ext4/lwext4，修复 `openat`、`read/write`、`getdents64`、`dup/dup2`、`getcwd`、`unlink`、`chdir` 等基础文件系统语义，并参与 main 与 MachineTest 分支的合并整理。相关证据包括 `documents/worklog.md`、`documents/ext4 迁移记录 2026-04-22.md`、`documents/ext4 迁移具体变更内容.md`、`documents/VFS 与 Dentry 实现原理.md`，以及 git 提交 `e7ec1269`、`d4929035`、`bb33997f`、`26e0c1af`、`73617245` 等。
2. **杜宇恒：内存管理、基础 syscall、网络系统与 LTP 推进**。杜宇恒主要负责 `brk`、`mmap`、`munmap`、`times`、`uname`、`nanosleep` 等系统调用实现，并承担网络子系统开发，包括 socket table/fd table 整合、`socket`、`bind`、`listen`、

connect、sendto/recvfrom、sendmsg/recvmsg、setsockopt/getsockopt 等接口。后期还负责 LTP 可用用例筛选和测试流程推进。相关证据包括 documents/worklog.md、documents/ltp 测试指南.md、documents/ltp 子集测试状态.md、documents/netperf 测试指南.md、documents/iperf3 实施计划.md，以及 git 提交 5668a9b0、d0f74a2f、131ab1b5、0320fd35、6e4a5efd、7ea976be、0373875d、73e2f054 等。

3. **宋天琦：BusyBox/glibc/LoongArch 适配、官方评测链路与 benchmark 修复**。宋天琦主要负责 BusyBox、glibc BusyBox 和 LoongArch64 适配，包括 ELF 加载、用户栈、COW fork、LoongArch 启动与页表、BusyBox 测试脚本、/proc 虚拟文件系统、标准设备和 libc 兼容 syscall。后期主要负责官方 MachineTest 链路，包括官方 sdcard 双盘挂载、libctest/pthread/futex 语义、libcbench 线程退出与共享地址空间、lmbench/iozone 问题排查，以及从 MachineTest 向 main 的迁移记录。相关证据包括 documents/busybox_loongarch 适配具体更改.md、documents/busybox 系统调用实现计划.md、documents/libctest 开发文档.md、documents/测试记录.md、documents/MachineTest 迁移记录 2026-06-16.md、documents/MachineTest 近期改动迁移说明 2026-06-29.md，以及 git 提交 7780213d、e8e2a8de、4ab30d40、80315eff、e3bbe9cd、a741cce7、0948d279、f81fb9c9 等。

4. **协作方式与质量控制**。团队采用“文档记录 + git 小步提交 + 本地/官方双重验证”的协作方式。每一类功能开发或问题排查通常先在 documents/worklog.md 或专题文档中记录现象、根因、修复方案和验证结果，再通过 git commit 固化。对于复杂分支合并和官方评测链路，团队额外在 documents/MachineTest 迁移记录 2026-06-16.md 和 documents/MachineTest 近期改动迁移说明 2026-06-29.md 中记录迁移来源、保留内容、未迁移内容和验证情况，从而保证开发过程可追溯、可复现。

综上，雷雨泽、杜宇恒、宋天琦三人分别在文件系统与项目统筹、内存/网络/LTP、架构适配与评测链路等方向形成分工协作；documents/worklog.md、各专题文档以及 git commit 历史共同构成了开发过程和成员贡献的证据链。

12 Scintilla 技术问题及解决方案示例

ScintillaOS 的问题定位以跨模块不变量为主线。一个表面上的系统调用失败，可能由地址空间权限、任务状态、目录项缓存、底层 Ext4 生命周期或设备中断时序共同造成。因此，问题处理不能只修正调用返回值，还要明确资源所有权、锁顺序、状态转换和可复现验证范围。本章选取 VFS、COW、mmap 与 SMP 阻塞唤醒四类代表性案例；其中的代码片段均对应当前源码基线。

12.1 问题记录

表 12.1 代表性技术问题与解决方案

问题类别	现象	根因分析	解决方案与验证
VFS/Ext4	双盘路径解析、并发创建或重命名时可能得到陈旧目录项	单核字符串路径和线性目录项缓存无法表达挂载边界、目录修改与缓存代际	以 Dentry、超级块、目录变更锁、正负项缓存和代际复核维护命名空间；以双盘与并发文件系统压力测试验证
内存/COW	并发 `fork` 后，内核向用户缓冲区写回可能偶发失败	页准备完成后，另一线程可将共享页重新写保护；跨核还可能保留陈旧 TLB	写回前准备所有用户页，失败时重新处理 COW 并重试；共享地址空间变化后执行远端 TLB 刷新
系统调用/mmap	`mmap(NULL, ...)`、`MAP_FIXED` 与共享地址空间并发时可能重叠或错误回收页面	选址、VMA 发布、溢出检查和失效页回收若分散执行，会与 `CLONE_VM` 线程竞争	将自动选址与 VMA 发布置于同一地址空间锁内；检查地址溢出，并在失效后刷新其他活跃 CPU 的 TLB

问题类别	现象	根因分析	解决方案与验证
SMP 阻塞唤醒与锁	wait、futex、pipe、poll 或网络 I/O 可能出现丢失唤醒或锁反转	“检查条件—登记等待—切出” 不是原子过程;唤醒方不能把仍在运行的上下文直接入队	以状态机和固定锁等级完成交接;使用多核定向与 BuildStorm 负载验证

12.2 案例一：VFS、双盘路径解析与并发目录项缓存

早期原型可以用当前工作目录字符串拼接出完整路径，再直接交给单个 Ext4 实例打开。这种方案无法表达挂载边界，也无法在多个 CPU 同时创建、删除或重命名同一目录中的文件时维护缓存一致性。当前实现仍保留用户可见路径 `path` 与底层文件系统路径 `fs_path` 的区分，但将底层 Ext4 的串行化和命名空间缓存明确纳入 VFS 对象。

当前每个已挂载文件系统由 `VfsSuperBlock` 描述。底层 lwext4 并不具有可重入性，因此不再依赖单核前提下的 `UPSafeCell`，而是使用 `FsLock` 保护单卷访问，并结合全局 lwext4 约束完成直接 I/O 的序列化：

```
pub struct VfsSuperBlock {
    pub fs_type: &'static str,
    lwext4_mount_path: String,
    pub fs_data: FsLock<FsInner>,
}

pub(crate) fn lock_fs(&self) -> VfsFsGuard<'_> {
    let local = self.fs_data.exclusive_access();
    let global = LWEXT4_GLOBAL.read();
    VfsFsGuard {
        _global: global,
        local,
    }
}
```

目录项结构不仅携带挂载后的内部路径和超级块，也维护父引用、目录子项、目录变更锁、缓存代际和负项缓存。这样，`openat`、`mkdirat`、`unlinkat` 和 `renameat` 可以在同一命名空间中工作，同时将慢速 Ext4 查找移出短时间的缓存临界区：

```
pub struct Dentry {
    pub name: String,
    pub path: String,
```

```
    pub fs_path: String,
    pub inode_num: u32,
    pub is_dir: bool,
    pub is_link: bool,
    pub parent: Option<Weak<Dentry>>,
    pub super_block: Arc<VfsSuperBlock>,
    runtime_size: Option<Arc<InodeRuntimeSize>>,
    pub children: SleepLock<Option<BTreeMap<String, Arc<Dentry>>>>,
    child_mutation: SleepLock<()>,
    child_cache_epoch: AtomicU64,
    parent_validation_epoch: AtomicU64,
    negative_children: SleepLock<Option<BTreeSet<String>>>,
    inode_info: DentryLock<Option<InodeInfo>>,
}
```

当前 vfs_lookup 的关键不再是“字符串命中即返回”，而是先读取稳定代际，检查正项和负项缓存；未命中时取得目录变更锁，再次复核缓存和代际，最后才进入 Ext4 查询。查询结果只在代际仍有效时发布，挂载点则通过 mounted_child 透明切换至另一超级块：

```
let lookup_epoch = parent.child_cache_epoch.load(Ordering::Acquire);
if lookup_epoch & 1 != 0 {
    let guard = parent.child_mutation.lock();
    drop(guard);
    continue;
}
if let Some(cached) = cached_positive_child(parent, name) {
    return Some(mounted_child(cached));
}
if negative_child_cached(parent, name, lookup_epoch) {
    return None;
}

let _lookup = parent.child_mutation.lock();
let lookup_epoch = parent.child_cache_epoch.load(Ordering::Acquire);
let info = parent
    .super_block
    .get_inode_info_result(&child_fs_path(parent, name))
    .ok()?;
let child = Dentry::new_child(parent, name, info.inode_num, info.is_dir,
    info.is_link);
child.set_cached_inode_info(info);
```

```
22 if let Some(canonical) = insert_positive_child(parent, child.clone(),
       Some(lookup_epoch)) {
23     return Some(mounted_child(canonical));
24 }
```

对于 /mnt/foo，外部 path 仍保持 /mnt/foo，而挂载根后的 fs_path 在测试盘中解析为 /foo；.. 则通过父 Dentry 与挂载根关系返回正确命名空间。决赛阶段还补充了目录变更期间的代际复核、打开文件 unlink 后的 orphan 生命周期、覆盖 rename 和 ftruncate 的运行时大小同步。验证同时覆盖双盘挂载、不同块设备上的差异文件、并发创建/重命名/删除、已打开 fd 在 rename 或 unlink 后继续访问，以及卸载后重挂载的数据一致性。

12.3 案例二：多核 COW、用户态写回与远端 TLB 刷新

写时复制仍是 fork 的核心机制：父子进程初始共享私有页，双方去除写权限，首次写访问触发缺页后再复制页面。但是，多线程和多核条件下还存在一个初赛阶段没有覆盖的窗口：内核已经为系统调用返回缓冲区准备了可写页，另一线程恰好执行 fork 并将该页重新写保护。若此时直接把内部竞争转换为 EFAULT，glibc、线程运行时和 Cargo 子进程会得到随机失败。

当前实现先通过 prepare_task_user_access 验证用户 VMA 并按页处理缺页或 COW；需要写入共享地址空间时，记录地址空间活动信息并在页表变化后刷新其他活跃 CPU 的 TLB。实际写回失败时并不立即向用户态报告 EFAULT，而是重新准备页面并重试：

```
1  pub(crate) fn copy_to_task_user(task: &Arc<TaskControlBlock>, addr: usize
       , src: &[u8]) -> bool {
2      let mut retries = 0usize;
3      loop {
4          if !prepare_task_user_access(task, addr, src.len(), true) {
5              return false;
6          }
7          let memory_set = task.inner_exclusive_access().memory_set.
               fault_view();
8          if memory_set.copy_to_user(addr, src) {
9              return true;
10         }
11
12         // A concurrent fork can write-protect the shared address space
               after
13         // fault preparation but before the VM-locked copy.
14         retries = retries.wrapping_add(1);
```

```
        if retries & 7 == 0 {
            crate::task::suspend_current_and_run_next_reason(
                crate::task::SuspendReason::CopyToUserRetry,
            );
        }
    }
}
```

远端 TLB 刷新同样必须与页帧回收解耦。地址空间保存活跃 CPU 掩码，撤销映射或改变共享页翻译后先执行跨核失效，再允许退役页帧回到分配器；否则另一个 hart 可能经由陈旧 TLB 访问已经复用的物理页。用户页准备路径中的相关调用如下：

```
if let Some(activity) = shared_activity.as_ref() {
    let _ = flush_remote_tlb(
        activity,
        shared_flush_start,
        shared_flush_end - shared_flush_start,
    );
}
```

该方案保留了 COW 的按需复制优势：只有真正写入共享页时才分配新帧；同时把跨核页表可见性和系统调用用户缓冲区写回纳入同一正确性边界。验证包括 `fork` 后父子隔离、`clone` 共享地址空间、`mprotect` 与 `munmap` 的跨核回归，以及八核 BuildStorm 中大量子进程、线程和状态结构写回的端到端构建。

12.4 案例三：`mmap` 的边界条件与共享地址空间并发

`mmap` 的兼容性不只是把一段虚拟地址加入 VMA 列表。当前实现首先检查页对齐、长度、保护位、映射类型、文件偏移和权限；对于 `MAP_FIXED`，还要延迟释放被替换映射的页帧，直到其他活跃 CPU 完成 TLB 刷新。对于不指定地址的匿名私有映射，自动选址和 VMA 发布必须是同一个事务，否则两个 `CLONE_VM` 线程可能选择到相同空洞。

当前 `sys_mmap` 使用溢出检查得到映射末端，并将 `mmap(NULL, ...)` 的自动选址委托给持有共享 VMA 锁的 `insert_lazy_area_auto`。这段逻辑替代了早期“取最大结束地址后在锁外映射”的实现：

```
let real_start = if start == 0 && map_shared {
    let max_end: usize = memory_set.max_end_vaddr().into();
    (max_end + crate::config::PAGE_SIZE - 1) / crate::config::PAGE_SIZE
        * crate::config::PAGE_SIZE
} else {
```

```
6       start
7   };
8   let Some(real_end) = real_start.checked_add(len) else {
9       return -22;
10  };
11
12  if !map_shared && start == 0 {
13      let Some(aligned_len) = page_align_up(len) else {
14          fail_after_fixed_unmap!(-12);
15      };
16      let Some(mapped_start) = memory_set.insert_lazy_area_auto(len,
            permission)
17      else {
18          fail_after_fixed_unmap!(-12);
19      };
20      let Some(mapped_end) = mapped_start
21          .raw()
22          .checked_add(aligned_len)
23          .map(polyhal::VirtAddr::from)
24      else {
25          fail_after_fixed_unmap!(-12);
26      };
27      sync_and_return!(
28          mapped_start.raw() as isize,
29          mapped_start,
30          mapped_end
31      );
32  }
```

地址空间内部的自动选址会先对长度页对齐，再在持有共享 VMA 锁时计算起始地址、检查重叠并发布惰性区域；因此不会把零地址直接解释为普通固定映射，也不会把选址与发布拆成可竞争的两个步骤。文件映射另外检查文件偏移和读写权限；MAP_FIXED_NOREPLACE 在重叠时返回 EEXIST，MAP_FIXED 则在新映射安装前撤销旧 VMA，并在退出该路径前完成退休页帧的远端 TLB 刷新。

本案例的验证不再只依赖基础 mmap 用例，而包括匿名/文件映射、MAP_PRIVATE 与 MAP_SHARED、mprotect、madvise、munmap、跨页用户缓冲区和 fork COW 的组合压力。八核定向测试和复杂构建均用来确认 VMA 修改、页故障处理与页帧回收没有形成并发重叠或陈旧翻译。

12.5 案例四：SMP 阻塞唤醒与锁顺序

单核下，“检查条件、把任务加入等待队列、切换到调度器”通常由中断关闭隐式串行化；多核下，另一个 CPU 可以恰好在任务切出前发布事件。若唤醒方直接把尚未保存上下文的任务插入就绪队列，同一任务可能被两个 CPU 使用；若唤醒发生在检查条件与登记等待之间，则任务可能永久错过唯一一次唤醒。ScintillaOS 因此将任务阻塞建模为显式状态机，并把调度器状态与等待队列登记置于同一临界区。

阻塞开始时，任务先在 `SchedulerLock -> TaskLock` 顺序下从 `Running` 变为 `Blocking`，再登记对应等待队列；此时仍在当前 CPU 上运行的上下文不会被重新入队：

```
let mut scheduler = SCHEDULER.exclusive_access();
if task.is_enqueued() {
    assert!(scheduler.remove(task.getpid()));
}
let mut inner = task.inner_exclusive_access();
debug_assert_eq!(inner.task_status, TaskStatus::Running);
inner.task_status = TaskStatus::Blocking;
inner.block_reason = reason;
let task_cx_ptr = &mut inner.task_cx as *mut _;
drop(inner);

match queue {
    BlockQueue::Net => scheduler.net_blocked.push(task.clone()),
    BlockQueue::WaitPid => scheduler.waitpid_blocked.push(task.clone()),
    BlockQueue::None => {}
}
```

若此时另一 CPU 提前唤醒任务，`wake_task_filtered` 只将状态改为 `WakePending`；真正的就绪队列发布推迟到当前 CPU 完成上下文保存之后。若任务已经完成切出而处于 `Blocked`，唤醒方才将其置为 `Ready`、选择目标 CPU 并发送重调度请求：

```
let enqueue_now = match inner.task_status {
    TaskStatus::Blocking => {
        inner.task_status = TaskStatus::WakePending;
        inner.block_reason = None;
        return true;
    }
    TaskStatus::Blocked => {
        inner.task_status = TaskStatus::Ready;
```

```
        inner.block_reason = None;
        true
    }
    TaskStatus::WakePending => return true,
    _ => false,
};
drop(inner);
let target = enqueue_now.then(|| scheduler.add_priority(task.clone()));
drop(scheduler);
if let Some(target) = target {
    super::processor::request_reschedule(target);
}
```

当前 CPU 回到 idle 上下文后，`complete_context_switch` 将 `Blocking` 完成为 `Blocked`，或将已被提前唤醒的 `WakePending` 完成为 `Ready` 并入队。这一延迟发布保证任何 CPU 恢复任务前，其内核上下文已经完整保存。

为避免修复唤醒问题时引入新死锁，内核还定义了领域锁的全序。futex 等待表必须先于调度器锁取得，使等待 key 与 `Blocking` 状态可以原子发布；调度器、文件表、任务、信号、虚拟内存、VFS、页缓存、页帧和设备依次排序。诊断构建会记录每个 hart 的持锁栈，并拒绝等级逆序、同等级地址逆序以及持领域锁进入调度或休眠：

```
pub enum LockClass {
    CpuLocal = 0,
    PidRegistry = 1,
    Futex = 2,
    Scheduler = 3,
    Files = 4,
    Task = 5,
    SignalActions = 6,
    Vm = 7,
    VmPages = 8,
    VmPageTable = 9,
    Vfs = 10,
    PageCache = 11,
    FrameRef = 12,
    FrameAllocator = 13,
    Fs = 14,
    FsGlobal = 15,
    Dentry = 16,
    FsAux = 17,
    Device = 18,
```

```
}
```

该协议被 `waitpid`、futex、管道、`poll/epoll`、网络 socket、超时和信号中断路径共同使用。四核 SMP 回归中的调度、亲和性、`clone`、futex 与内存权限测试，以及双架构八核 CAgent 和 BuildStorm 构建，分别检查局部状态不变量和真实并发工作负载；锁诊断记录则用于定位锁反转、遗漏唤醒和持锁切换等问题。

13 构建、评测与结果

13.1 构建与运行方法

ScintillaOS 根目录的 `Makefile` 提供如下常用目标：

- `make all`：生成 RISC-V、LoongArch 内核及根文件系统镜像；
- `make kernel ARCH=riscv64`：仅编译 RISC-V 内核；
- `make fs-img ARCH=riscv64`：构建包含用户程序的 Ext4 镜像；
- `make run ARCH=riscv64`：单盘开发模式运行；
- `make run-dual ARCH=riscv64 TEST_DISK=<测试盘>`：双盘方式模拟评测环境。

13.2 测试框架

ScintillaOS 运行在官方提供的 docker 环境中，由 `initproc` 在双盘模式下挂载测试盘，扫描并运行官方测试盘中的 `testcode.sh` 测试脚本。测试输出使用统一的开始/结束标记，外部裁判脚本据此提取测试组结果。

14 总结与展望

14.1 当前工作情况

ScintillaOS 已由 rCore 教学内核演进为面向 QEMU 双架构竞赛环境的 SMP 操作系统内核。系统以 Rust 实现，支持 RISC-V 与 LoongArch，当前配置分别最多支持 8 个和 12 个应用 CPU。围绕 Linux ABI 用户程序，内核提供进程与线程、COW、按需虚拟内存、VFS/Ext4 双盘、VirtIO 块设备与网络、信号和 futex，以及 eventfd、epoll、管道、Unix 域 socketpair 和 TCP/UDP socket 等运行时所需对象。

14.2 经验总结

在 ScintillaOS 的开发中，我们遇到了不少问题与挑战，也由此积累了许多开发经验。

第一，操作系统模块的接口正确并不足以保证系统正确。文件描述符、任务状态、VMA、Dentry、页帧和设备队列都具有跨模块生命周期，设计时应先明确谁拥有对象、何时转移所有权、何时允许回收，再讨论局部接口的实现。打开文件后的 unlink、共享地址空间、COW 与远端 TLB 刷新均说明这一点。

第二，多核改造的核心是协议而不是给旧全局数据结构加锁。调度切换、等待队列、唤醒、页表失效和锁获取必须有可描述的状态转换与顺序。`Blocking` 到 `WakePending` 的交接、`SchedulerLock -> TaskLock` 的锁序以及调试构建中的 lockdep，使“偶发卡死”能够被化为可验证的不变量和可定位的反例。

第三，真实复杂负载比单项微测试更能揭示内核边界。BuildStorm 同时施压线程运行时、`fork/exec`、futex、文件元数据、页缓存、Ext4、内核堆和用户缓冲区回写；但复杂负载的耗时也极易受到宿主资源影响。因此，团队将短测用于缩小根因，将干净双盘全量日志用于结算功能结果，并区分内核故障、覆盖层污染和宿主资源终止。

第四，可追溯的验证过程与修复本身同等重要。将研发分支与自动评测分支分离、保存测试盘版本和 QEMU 参数、使用干净 qcow2 覆盖层、保留失败样本和最终串口标记，使结论能够被复现、审阅和在后续提交中重新确认。

14.3 未来展望

后续工作将沿三个方向继续推进。其一，在最终提交二进制上重新执行完整官方链路，并为更高核数、长时间网络负载和复杂构建保留稳定的回归矩阵；同时继续改进每 CPU 队列之间的负载均衡、迁移策略和诊断能力。其二，继续完善

文件系统的并发元数据路径、页缓存回收、写回压力控制和崩溃恢复语义，减少构建型负载下的锁竞争与内存占用。其三，扩展 Linux/POSIX 兼容范围，包括更多 LTP 子集、网络协议与 socket 选项、可观测性接口和安全边界测试。

这些优化将继续遵循现有原则：以通用内核语义而非测例特判解决问题；在目标架构、双盘、保留设备和真实用户态负载下验证；并将源码版本、测试条件、反例和结论一同归档。

附录 A　AI 辅助使用说明

ScintillaOS 在开发过程中使用了生成式 AI、AI Agent 和智能 IDE 辅助工具。主要使用的工具包括 deepseek v4 pro、Codex 类 AI Agent，以及编辑器中的智能补全能力。AI 工具并不只用于文档润色，也参与了代码阅读、问题定位、局部实现草案生成、补丁修改建议、测试日志分析和报告整理等工作。团队在使用 AI 的过程中保留了主要交流记录，并通过 `documents/worklog.md`、各专题文档以及 git 提交历史记录具体功能的实现过程、问题原因和验证结果。

在系统设计层面，ScintillaOS 的总体技术路线由团队确定，即在 rCore 教学内核的基础上继续扩展，面向 QEMU 官方评测环境实现 Linux ABI 兼容能力。系统采用 main 分支作为主要开发分支，MachineTest 分支作为官方评测分支；这一分支策略、双盘评测模型、VFS/Dentry/Mount 架构、Ext4/lwext4 迁移、进程与地址空间组织、官方测试链路安排等关键设计，均由团队成员结合比赛要求和本项目实际情况确定。AI 在这一过程中主要帮助整理方案表述、归纳模块依赖关系、提示可能的风险点，但最终设计取舍由团队成员完成。

在代码实现层面，AI 参与了较多局部开发工作。例如，在文件系统、进程管理、内存管理、futex、信号、测试 runner、评分输出格式、README 和部分文档整理中，AI 曾根据已有代码生成局部补丁、提出修改方案或帮助重构代码片段。部分代码最初由 AI 给出草案，再由队员根据项目架构进行人工修改、合并和验证；也有部分代码由队员先写出主体，再使用 AI 辅助检查边界条件、错误码、日志输出和文档说明。因此，最终代码并非完全人工逐字编写，也不是 AI 独立生成后直接提交，而是经过队员理解、筛选、改写和测试后的结果。

在测试和调试层面，AI 对项目帮助较大。团队曾使用 AI 辅助分析 QEMU 串口输出、panic 日志、官方 judge 脚本行为、LTP 输出格式、libctest 超时原因、libcbench 线程退出卡死、lmbench 和 iozone 的性能与文件系统问题等。相关过程可以在 `documents/libctest 开发文档.md`、`documents/ltp 测试指南.md`、`documents/ltp 子集测试状态.md`、`documents/测试记录.md`、`documents/MachineTest 迁移记录 2026-06-16.md` 和 `documents/MachineTest 近期改动迁移说明 2026-06-29.md` 中看到。AI 在这些场景中提供了排查方向和候选解释，但是否采纳、如何修改、是否回退以及是否进入最终评测版本，均由团队通过本地 Docker/QEMU 测试和官方评测结果判断。

AI 生成内容并不总是正确。开发过程中曾出现过 AI 对官方评分脚本理解不完整、对系统调用语义判断过于简化、忽略 LoongArch 与 RISC-V 架构差异、建议的超时策略不适合最终评测、生成的代码与现有借用关系或锁顺序冲突等问题。团队发现这些问题后，会通过阅读源码、比对 Linux 语义、重新运行测试、查看

git diff 和分析日志来修正。对于可能影响核心语义的修改，例如 COW、页表回收、robust futex、clear_child_tid、VFS 路径解析、pipe 阻塞唤醒和 socket fd 管理，团队不会只依据 AI 输出判断正确性，而是结合测试结果和代码审查决定是否保留。

在文档整理阶段，AI 也参与了较多工作。团队使用 AI 根据已有源码、开发日志和专题文档整理报告内容，包括系统总体架构、模块边界、团队分工、libctest 开发说明、README、总结与展望等。文档中的技术结论主要来自仓库代码、git 记录、官方评测结果和本地测试日志；AI 的作用是帮助组织语言、提炼结构和减少遗漏。若 AI 生成的表述与实际代码或测试结果不一致，团队以实际代码和测试结果为准，并对文档进行人工修正。

最终提交版本的设计逻辑和技术细节由团队掌握。团队成员能够解释 ScintillaOS 的启动流程、系统调用分发、进程和线程模型、地址空间管理、COW 与按需分页、VFS/Dentry/Mount 结构、Ext4 双盘访问、futex 与信号处理、socket fd 统一管理以及官方评测链路。AI 工具在项目中承担的是辅助开发、辅助分析和辅助表达的角色，而不是替代团队完成方案设计、核心理解、最终调试和提交决策。对于最终代码中的 AI 参与部分，团队已经通过人工修改、构建检查、本地运行和官方评测进行了确认，并能够在需要时独立继续修改和调试。

对于任何由 AI 协助编写或修改的内容，团队成员均应完成以下确认：第一，对照实际源代码逐行审阅，确认函数名、数据结构和控制流未被误述；第二，在目标架构和 QEMU 环境中重新编译、运行对应测试；第三，在 `documents/worklog.md` 或版本控制记录中保留功能变更、责任人和验证证据。特别是内核中的 `unsafe`、页表、用户指针、并发控制和设备 I/O 路径，不能仅依据 AI 生成文本判断正确性。

本报告中的设计取舍、支持状态和已知限制以仓库代码、提交记录、测试日志及小组人工审核为准。若最终提交版本与报告中的代码摘录不一致，应以最终代码与重新生成的测试结果更新本报告。

附录 B　参考资料

1. RISC-V International，*The RISC-V Instruction Set Manual*。
2. lwext4 文档与源码。